



6CCS3PRJ Final Year
Preemptive Prediction of LLM Behavior
Using Balanced Conformal Prediction

Final Project Report

Author: Fathima Maryam Risvi

Course: Bsc (Hons) Computer Science

Student ID: K23098274

Supervisor: Dr. Nicola Paoletti

April 23, 2026

Originality Avowal

I verify that I am the sole author of this report, except where explicitly stated to the contrary. I grant the right to King's College London to make paper and electronic copies of the submitted work for purposes of marking, plagiarism detection and archival, and to upload a copy of the work to Turnitin or another trusted plagiarism detection service. I confirm this report does not exceed 25,000 words.

Fathima Maryam Risvi

April 23, 2026

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr Nicola Paoletti. His insightful feedback, coupled with his experience in the field, has been a key factor in shaping the success of this project. His encouraging words were a great source of motivation, inspiring me to push forward and constantly refine my work to a high standard. I deeply appreciate the time and effort he dedicated to helping me overcome challenges and guiding me towards a successful outcome. Finally, I would like to thank my family for their unwavering love, encouragement, and support throughout my studies. Their belief in me has been a constant source of strength and motivation.

Abstract

Large language models (LLMs) are widely used in safety-critical applications, yet failures such as hallucinating answers to unanswerable questions, ignoring instructions, or generating harmful content instead of appropriate responses are often detected only after full response generation, wasting time and computation. This project reproduces and extends the methodology of Ashok and May from *Language Models Can Predict Their Own Behaviour* (2025), using linear probes trained on Llama-3.2-3B hidden states to predict model behaviour before any output tokens are generated. Conformal prediction is a method that provides statistical confidence guarantees for model predictions by quantifying uncertainty; however, it often achieves high reliability by abstaining on uncertain cases, which can significantly reduce practical coverage. To address this, four new conformal decision strategies are introduced to better handle uncertainty in safety-critical settings, with the aim of reducing missed failures and improving real-world deployability.

Contents

1	Introduction	2
2	Background	5
2.1	The Critical Problem of Language Model Misbehavior	5
2.2	How Language Models Generate Responses: The Architecture And Hidden States	7
2.3	Probing Language Model Representations	12
2.4	Conformal Prediction	20
2.5	Conformal Risk Control	24
3	Literature Review	27
3.1	Refusal in Language Models Is Mediated by a Single Direction	27
3.2	The Geometry of Truth: Emergent Linear Structure in LLM Representations of True/False Datasets	29
4	Language Model Behavior Prediction Framework	31
4.1	Problem Statement	31
4.2	Methodology	32
4.3	Dataset & Tasks	33
4.4	Pipeline	35
5	Own Implementation	37
5.1	Tech Stack	37
5.2	Code Walkthrough	39
5.3	Methods and Design Decisions	43
6	Results	51
6.1	Experimental Setup	51
6.2	Method 1 : Baseline Probe Performance	53
6.3	Method 2 : CP Threshold (Paper’s Core Method)	54
6.4	Method 3a : Singleton + Conservative Rule	56
6.5	Method 3b : Class-Conditional Singleton + Abstain Rule	57
6.6	Method 4 : Conformal Risk Control	59
6.7	Method 5 : Misbehaviour Direction + CP Rule	60
6.8	Cross-Method Comparison at Alpha = 0.91	62
6.9	Visualizations	64

7	Legal, Social, Ethical and Professional Issues	68
7.1	Legal Issues	68
7.2	Ethical and Social Issues	68
7.3	Professional Issues	69
7.4	BCS Code of Conduct	69
8	Conclusion and Future Work	70
9	References	72

Chapter 1

Introduction

Large language models (LLMs) have become central to modern AI systems, powering applications from chatbots to autonomous reasoning tools. Yet despite their capabilities, they remain unreliable in important ways; they can hallucinate answers to questions that have no answer, refuse questions they should answer, or produce harmful outputs altogether. In safety-critical deployments, these failures carry real consequences.

A fundamental inefficiency compounds this problem: failures are typically caught only *after* the model has finished generating its full response. The system spends time and energy producing an output before anyone can determine whether it was safe to show. As models grow larger and outputs longer, this cost scales accordingly. The natural question is whether failure can be anticipated *before* generation begins.

Recent work by Ashok and May (*Language Models Can Predict Their Own Behaviour*, 2025) suggests it can. Their approach trains a lightweight linear probe on a model’s internal hidden states, the intermediate vector representations produced while processing an input, to predict whether the model will behave correctly, without ever running the decoder. This project reproduces their pipeline on Llama-3.2-3B using the SELF-AWARE benchmark, a dataset of human-annotated unanswerable questions that tests whether the model correctly recognises the limits of its own knowledge. The probe reads activations at the middle transformer layer (layer 14 of 28) and outputs a probability that the model will respond correctly — all before a single output token is generated.

Conformal prediction is then layered on top to provide statistically rigorous guarantees on those probe outputs. Rather than trusting a raw probability, conformal prediction calibrates a threshold such that the system’s accuracy on the examples it acts on is provably bounded.

The cost is coverage: to guarantee high accuracy, the system must sometimes abstain and a system that abstains on most inputs provides little practical value, however accurate it is on the remainder.

To make this concrete, consider a simple deployment scenario :

A user asks: “*What is the capital of France?*”, an answerable question. Before the LLM writes a single word of its response, its internal hidden states are extracted and fed to the probe. The probe outputs two probabilities:

$P(label1)$ — Model is about to respond correctly Correct limit awareness: answered when answerable, abstained when unanswerable $P(label0)$ — Model is about to misbehave Incorrect limit awareness: hallucinated to an unanswerable question or refused an answerable one
--

A decision is then made before the output is ever shown to the user: pass the response through, or block it with a warning. The four methods each implement a different rule for making that call.

Method 3a : Singleton + Conservative Rule

Unless the probe is fully confident the model will behave correctly (label 1), the output is blocked. This method makes no abstentions, as every case is either passed or blocked.

Method 3b : Class-Conditional Singleton + Abstain Rule

Takes a more balanced approach, using separate calibration per class to better capture uncertainty. The probe only intervenes when it is confident in either correct or incorrect behaviour; when uncertainty remains, it abstains and allows the model’s output to pass through rather than enforcing a potentially unreliable decision.

Method 4 : Conformal Risk Control

Shifts the focus from accuracy to a direct safety constraint, aiming to ensure that the proportion of missed failures stays below a user-defined limit. Each input is evaluated by checking whether the predicted probability of misbehaviour exceeds a calibrated threshold; if it does, the response is blocked, otherwise it is allowed to pass through.

Method 5 : Misbehaviour Direction + CP Rule

Rather than using the probe’s output probabilities, this method projects each input’s hidden

states onto a single score that reflects how closely the model’s internal representations resemble past misbehaving examples. Conformal prediction sets are built from this score, and decisions follow a conservative blocking policy where only confident correct cases are allowed through, while all others are blocked.

Together, these methods reframe the central question from “*How accurate is the probe when it predicts?*” to “*Can a model’s own internal states provide actionable, reliable warnings on a substantial fraction of inputs, without missing the failures that matter most?*”

Overall, this work contributes to the development of more reliable and practical safety mechanisms for language models. It demonstrates how internal model activations can be leveraged to provide earlier and more reliable warnings of potential failures, moving toward systems that can detect risks before generating full responses.

Chapter 2

Background

2.1 The Critical Problem of Language Model Misbehavior

Language models (LMs) have rapidly transitioned from research curiosities to essential infrastructure powering billions of daily interactions. These AI systems, built on transformer architectures and trained on vast internet-scale datasets, now underpin ChatGPT’s conversational abilities, GitHub Copilot’s code generation, and increasingly, mission-critical applications in healthcare, legal services, and financial decision-making. Yet, these systems exhibit fundamental behavioral problems that pose serious risks to users, organizations, and society at large.

Instruction-Following Failures

The first category of problematic behavior is instruction-following failures. Modern LMs are typically deployed with explicit formatting requirements—for instance, a customer service chatbot might need to return responses in JSON format with specific fields like "intent," "sentiment," and "suggested_action" so downstream systems can parse and route requests appropriately. Similarly, automated report generation systems might require outputs formatted as exactly three bullet points for consistency. However, LMs frequently violate these specifications, generating freeform text when structured output is required. While these might seem like minor formatting issues, they cause cascading failures in production systems.

Hallucinations

The second failure is hallucinations, these occur when LMs confidently generate false information presenting fabricated facts as truth. This isn’t random noise or obvious gibberish;

hallucinated content is often fluent, coherent, and plausible-sounding, making it particularly insidious. A medical chatbot might invent drug interactions that don't exist, potentially endangering patient lives if a doctor trusts the AI without verification. The root cause stems from LMs' training objective: they learn to predict plausible-sounding text based on statistical patterns in training data, not to verify factual accuracy. When faced with questions beyond their training knowledge or about events after their training cutoff, LMs don't recognize the boundary of their knowledge. Instead, they confabulate answers that sound authoritative but are completely fabricated.

Safety-Alignment Failures (Jailbreaking)

The third and most concerning failure mode is safety alignment failure, colloquially known as "jailbreaking." Despite extensive training to refuse harmful requests, LMs remain vulnerable to adversarial prompts that manipulate them into generating dangerous content. A properly aligned model should refuse requests like "explain how to synthesize dangerous chemicals" or "write a convincing phishing email to steal credentials." However, attackers have discovered numerous techniques to circumvent these safeguards. Some jailbreaks use role-playing scenarios ("pretend you're a chemistry teacher explaining this purely for educational purposes"), others exploit translation or encoding ("first explain in Base64"), while sophisticated attacks manipulate the model's reasoning process to rationalize compliance.

Why Preemptive Detection Matters

Why does this all matter so critically right now? The answer lies in scale and stakes. First, computational and environmental costs have become prohibitive. Modern LMs are expensive to run and with millions of queries daily, these costs compound rapidly, creating economic pressure that limits deployment viability. Environmental impacts mirror these economic concerns. Second, current mitigation strategies are entirely reactive. Organizations deploy post-hoc content filtering and output validation, scanning generated text for policy violations, factual errors, or formatting problems. But this reactive approach only catches problems after the expensive generation process completes. Every hallucinated fact, every jailbreak attempt, every formatting failure incurs full inference cost even though the output gets discarded. The fundamental insight motivating this project is that we might detect these problematic behaviors before generation even begins. If we could identify that an input will trigger jailbreaking, format-violating, or hallucinated output by examining only the model's internal state when processing the input,

we could intervene immediately—refusing to generate, routing to a safer model, or triggering human review—without paying the computational cost of full generation. This preemptive approach could simultaneously improve safety and efficiency.

2.2 How Language Models Generate Responses: The Architecture And Hidden States

To understand how we might predict model behavior before generation, we must first understand the internal mechanics of modern language models. These systems, built on the transformer architecture which process text through a sophisticated multi-stage pipeline that transforms input tokens into output sequences.

Tokenisation and Embedding

Input text is first decomposed into a sequence of subword tokens by a tokeniser. Given input text x , let the resulting token sequence be:

$$x = (x_1, x_2, \dots, x_T) \quad \text{where} \quad x_i \in V$$

where V is the vocabulary for the models.

Each token x_i is mapped to a d -dimensional embedding vector through a learned embedding matrix E , which provides the model’s initial representation of the input tokens. These embeddings capture basic lexical information but do not yet contain any contextual understanding.

$$E \in R^{|V| \times d}$$

For a given token x_i , its corresponding embedding vector is obtained by looking up its associated row in E :

$$e_i = E[x_i] \in R^d$$

where the resulting vector e_i is a numerical representation of the token in d -dimensional space.

Since embeddings alone do not contain any information about word order, positional information must also be introduced so the model can distinguish between sequences where the

same tokens appear but in different positions. In the original Transformer architecture, this is done by adding a positional encoding vector $p_i \in R^d$ directly to each token embedding e_i . Many modern models, such as LLaMA, instead use Rotary Position Embeddings (RoPE), which incorporate positional information inside the self-attention mechanism itself rather than by simple vector addition. RoPE modifies how attention scores are computed so that relative token positions are naturally preserved during attention.

Conceptually, the initial hidden state at layer 0 for each position can be represented as:

$$h_i^{(0)} = e_i + p_i \in R^d$$

where $h_i^{(0)}$ represents the input to the first Transformer layer, combining both the token’s semantic meaning and its positional information.

The Transformer Layer

The core of modern LMs is the transformer layer, which is applied L times in sequence. Each layer l computes a new hidden state from the previous one. Formally, a transformer layer consists of two main sub-layers: multi-head self-attention (MHSA) and a position-wise feed-forward network (FFN), with residual connections and layer normalisation applied to both.

Multi-Head Self-Attention

Self-attention allows each token to aggregate information from all other tokens in the sequence. Instead of processing words independently, each token builds a context-aware representation by deciding which other tokens are relevant to it.

To do this, every token is projected into three different roles: a query, a key, and a value. The query represents what the token is trying to find, the key represents what information it offers to others, and the value is the actual information it contributes if selected. Given the hidden states from the previous layer :

$$H^{(l-1)} = [h_1^{(l-1)}, \dots, h_T^{(l-1)}] \in R^{T \times d},$$

each attention head $k \in \{1, \dots, N_{heads}\}$ projects these into a lower-dimensional subspace via learned matrices $W_k^Q, W_k^K, W_k^V \in R^{d \times d_h}$. These projections produce the query, key, and value representations for each head:

$$Q_k = H^{(l-1)} W_k^Q, \quad K_k = H^{(l-1)} W_k^K, \quad V_k = H^{(l-1)} W_k^V,$$

where $d_h = d/N_{heads}$ is the per-head dimension. Using multiple heads allows the model to attend to different types of relationships in parallel.

Each head produces its output by computing a weighted sum of value vectors, where the weights are determined by the similarity between queries and keys using a scaled dot-product attention mechanism:

$$Attn_k(Q_k, K_k, V_k) = softmax\left(\frac{Q_k K_k^\top}{\sqrt{d_h}}\right) V_k \in R^{T \times d_h}.$$

The $\sqrt{d_h}$ scaling prevents the dot products from growing large in magnitude and destabilising the softmax. The outputs of all heads are concatenated and projected back to dimension d :

$$MHSA(H^{(l-1)}) = Concat(Attn_1, \dots, Attn_{N_{heads}}) W^O \in R^{T \times d},$$

where $W^O \in R^{d \times d}$ is a learned output projection.

Finally, a residual connection and layer normalisation are applied to produce the updated hidden states:

$$\tilde{h}_i^{(l)} = LayerNorm\left(h_i^{(l-1)} + MHSA(H^{(l-1)})_i\right).$$

Feed-Forward Network

While self-attention allows tokens to exchange information with each other, the feed-forward network (FFN) operates independently on each token. It is applied position-wise, meaning every token is processed using the same small neural network without interacting with others at this stage. This adds additional representational power beyond attention.

The FFN is a simple two-layer neural network:

$$FFN(z) = W_2 \cdot \sigma(W_1 z + b_1) + b_2,$$

where $W_1 \in R^{d_{ff} \times d}$ expands the representation to a wider intermediate dimension $d_{ff} = 4d$, and $W_2 \in R^{d \times d_{ff}}$ projects back down to d . The activation function σ is typically GeLU or

SiLU in modern models.

A second residual connection and layer normalisation give the final hidden state for layer l :

$$h_i^{(l)} = \text{LayerNorm}\left(\tilde{h}_i^{(l)} + \text{FFN}(\tilde{h}_i^{(l)})\right).$$

This full computation, MHSA followed by FFN each wrapped in a residual connection and layer normalisation, constitutes one Transformer layer, repeated L times with independent weights at every layer.

What Hidden States Encode Across Layers

At each layer l , the model maintains a hidden state for each token position i .

$$h_i^{(l)} \in R^d$$

These vectors are the internal representations that encode everything the model has “understood” about each token in context up to that stage of processing. As depth increases, the nature of these representations changes systematically.

- **Early layers** ($l \approx 1-8$): Primarily capture surface-level syntactic features, including token identities, part-of-speech information, local phrase structure, and character-level patterns.
- **Middle layers** ($l \approx 8-24$): Develop increasingly abstract semantic representations. The model begins to encode relational knowledge (for example, that “Paris” and “France” are linked by the capital-of relation), semantic similarity, and coreference.
- **Late layers** ($l \approx 24-L$): Representations become specialised for the model’s training objective, namely next-token prediction. These layers encode task-specific features and are most predictive of the output distribution.

Autoregressive Generation

Once the model has processed all T input tokens through all L layers, it begins generating output autoregressively, one token at a time. The final hidden state of the last input token $h_T^{(L)}$ is passed through an unembedding matrix

$$U \in R^{|V| \times d}$$

followed by a softmax to produce a probability distribution over the vocabulary:

$$P(x_{T+1} \mid x_1, \dots, x_T) = \text{softmax} \left(U h_T^{(L)} \right)$$

The next token x_{T+1} is then sampled from this distribution (or selected as the argmax under greedy decoding). This token is appended to the sequence, and the entire forward pass is repeated to generate subsequent tokens:

$$P(x_{T+t} \mid x_1, \dots, x_{T+t-1}) = \text{softmax} \left(U h_{T+t-1}^{(L)} \right)$$

This autoregressive process continues until a stopping criterion is reached, such as the generation of an end-of-sequence token or a maximum length constraint.

Note that processing the input requires a single forward pass regardless of prompt length T , while generating a response of length G requires G additional forward passes. For a model such as *LLaMA* – 3.1 – 8B with $L = 32$ layers, generating a 100-token response therefore requires 100 complete forward passes through all 32 layers, making output generation the dominant computational cost.

Here emerges the key insight: by the time the model finishes processing the input, the final hidden state contains a rich, compressed representation encoding everything the model “understands” about the task. If this hidden state already contains signals about problematic behaviors such as whether the model will violate format specifications, generate hallucinated content, or comply with a malicious request; we could potentially detect those signals before any generation occurs. This is the theoretical foundation for the probing approach: the model’s internal state at the end of input processing might already “know” how generation will unfold, even before the first output token appears.

2.3 Probing Language Model Representations

Probing is a methodology for investigating what information is encoded in the internal representations of neural networks. The central premise is simple: if a network’s hidden states encode some property (e.g. sentiment, syntactic role, or intent to comply with a harmful request), then a simple classifier trained on those hidden states should be able to recover that property. This technique was formalised by Alain and Bengio (2018) and has since become a standard tool in mechanistic interpretability.

In this project, we train probes to predict behavioural outcomes, from its hidden states at the end of input processing. We evaluate four distinct probe architectures, each making different assumptions about the structure of the relevant information.

2.3.1 Formal Setup

Let

$$H = [h_1, h_2, \dots, h_T] \in R^{T \times d}$$

denote the matrix of hidden states from a chosen layer l for a sequence of T tokens. A probe is a function:

$$p_\theta : R^{T \times d} \rightarrow [0, 1]$$

parameterised by θ , that maps the full sequence of hidden states to a scalar probability that the input will trigger the target behaviour (e.g. misbehave). The probe is trained using binary cross-entropy loss on a labelled dataset of hidden state and outcome pairs:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N [y_i \log p_\theta(H_i) + (1 - y_i) \log (1 - p_\theta(H_i))]$$

where $y_i \in \{0, 1\}$ is the observed behavioural label for example i .

The central question is: what aggregation strategy should be used to reduce the $T \times d$ hidden state matrix to a single probability? We consider four approaches.

2.3.2 Probe Architecture 1: Mean Centroid (MeanModel)

The simplest approach to classification in representation space is the nearest-centroid (mean) classifier. For each class $c \in \{0, 1\}$, we compute the centroid of all training hidden states belonging to that class.

Given a pooling strategy that reduces H to a single vector (e.g. by taking the mean across the sequence dimension), we compute:

$$\mu_c = \frac{1}{|D_c|} \sum_{i \in D_c} \text{pool}(H_i) \quad \text{for } c \in \{0, 1\}$$

where D_c is the set of training examples with label c , and $\text{pool}(H)$ denotes the mean-pooled hidden state:

$$\text{pool}(H) = \frac{1}{T} \sum_{t=1}^T h_t \in R^d$$

At test time, a new example is classified by its distance to each centroid:

$$\hat{y} = \arg \min_{c \in \{0, 1\}} \|\text{pool}(H_{test}) - \mu_c\|_2$$

The predicted probability is computed as a soft assignment based on normalised distances.

This approach requires no optimisation and is extremely fast to train.

Its implicit assumption is that the two classes are linearly separable in representation space and form roughly spherical clusters around their respective centroids.

2.3.3 Probe Architecture 2: Logistic Regression (Linear Probe)

The standard linear probe applies logistic regression to the pooled hidden state. This is the most widely used probe architecture in the interpretability literature.

After mean-pooling the hidden states to produce a vector

$$z = \text{pool}(H) \in R^d$$

the probe computes the following with learned weight vector $w \in R^d$ and bias $b \in R$.

$$p_\theta(H) = \sigma(w^T z + b) \quad \text{where} \quad \sigma(x) = \frac{1}{1 + e^{-x}}$$

The parameters $\theta = \{w, b\}$ are estimated by minimising the regularised binary cross-entropy:

$$\mathcal{L}(w, b) = - \sum_i \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right] + \lambda \|w\|_2^2,$$

For each probe example i , the model assigns a probability $\hat{y}_i = \sigma(w^\top z_i + b)$ to the positive class. The loss penalises confident wrong predictions: if the true label is $y_i = 1$ but $\hat{y}_i$ is small, the term $\log \hat{y}_i$ becomes large, and symmetrically for $y_i = 0$.

Summing over all examples: where for each example exactly one of the two terms is active: the first term contributes when $y_i = 1$, and the second when $y_i = 0$. The L_2 regularisation term $\lambda \|w\|_2^2 = \lambda \sum_j w_j^2$ penalises large weights, preventing the probe from overfitting and encouraging a parsimonious linear decision boundary. The coefficient $\lambda > 0$ controls the strength of this penalty. The simplicity of this model is a feature rather than a limitation: the deliberate use of a linear classifier ensures that any predictive signal discovered must be directly linearly accessible in the representation space. A complex non-linear probe could achieve high accuracy by performing substantial additional computation itself, making it unclear whether the information was genuinely encoded in the hidden states.

The geometric interpretation is particularly useful. Logistic regression learns a hyperplane

$$w^\top z + b = 0$$

in the d -dimensional representation space, classifying examples on one side as high-risk ($p > 0.5$) and examples on the other as low-risk ($p < 0.5$). A high-performing linear probe therefore implies that the property of interest is encoded as a roughly linearly separable direction in representation space.

2.3.4 Probe Architecture 3: Random Forest

The Random Forest probe relaxes the linearity assumption by using an ensemble of decision trees trained on the pooled hidden state

$$z \in R^d$$

A random forest consists of N_{trees} decision trees, each trained on a bootstrapped sample of the training data and a random subset of features at each split.

The prediction is the majority vote (or average probability) across all trees:

$$p_{RF}(H) = \frac{1}{N_{trees}} \sum_{k=1}^{N_{trees}} p_k(pool(H))$$

where p_k is the probability estimate from the k -th decision tree.

By averaging across many trees trained on different data subsets and feature subsets, the random forest reduces variance while maintaining low bias. Each tree partitions the d -dimensional feature space into axis-aligned hyperrectangles and assigns a class probability to each region. The ensemble can therefore capture non-linear and interaction effects in the representation space that a linear probe cannot, at the cost of reduced interpretability and higher computational overhead during training.

Random forests are known to perform well on high-dimensional data without extensive hyperparameter tuning, making them a useful middle ground between the simplicity of logistic regression and the expressivity of deep networks. A key diagnostic use of the random forest is feature importance: by tracking which dimensions of the hidden state appear most frequently in tree splits and produce the largest reductions in impurity, we can identify the subspace of the representation most relevant to the target behaviour.

2.3.5 Probe Architecture 4: Multi-Layer Perceptron (MLP)

The MLP probe fully relaxes the linearity assumption by using a small feedforward neural network as the classifier. This provides maximum expressivity at the cost of interpretability.

The architecture used in this project consists of two hidden layers:

$$z^{(1)} = ReLU(W_1 \cdot pool(H) + b_1) \quad W_1 \in R^{1000 \times d}$$

$$z^{(2)} = ReLU(W_2 \cdot z^{(1)} + b_2) \quad W_2 \in R^{500 \times 1000}$$

$$p_{MLP}(H) = \sigma(w_3^T \cdot z^{(2)} + b_3) \quad w_3 \in R^{500}$$

The parameters are trained using the Adam optimiser with mini-batch gradient descent: rather than computing gradients over the full dataset, weights are updated iteratively on small random batches, which is both faster and more stable in practice. Dropout regularisation is applied after each hidden layer during training, randomly zeroing a fraction of activations to prevent the network from over-relying on any single feature and thereby reducing overfitting.

However, because the MLP can learn non-linear functions of the hidden states, high accuracy is harder to interpret than in the linear case. The probe may achieve good performance by performing its own non-trivial computation, rather than simply reading off structure that is already explicit in the representation.

For this reason, the MLP probe is best interpreted as an *upper bound* on the predictability of the target behaviour from a given representation: strong MLP performance tells us the information is *present somewhere* in the hidden states, but says little about how it is structured. The linear probe, by contrast, is a more stringent test of whether that information is explicitly and directly accessible.

2.3.6 Aggregation Strategies Across the Sequence

All four probe architectures require reducing the sequence of hidden states

$$H \in R^{T \times d}$$

to a fixed-size representation. This sequence aggregation is a design choice that can significantly affect probe performance. We consider several strategies.

Mean Pooling

The simplest approach averages hidden states across all token positions:

$$pool_{mean}(H) = \frac{1}{T} \sum_{t=1}^T h_t$$

This treats all tokens as equally informative, which is appropriate when the property of interest is distributed across the entire input (for example, the overall topic or register of a message). However, it may dilute the signal from the specific tokens most indicative of the target behaviour.

Last-Token Pooling

Due to the causal (left-to-right) nature of Transformer attention, the final token’s hidden state has attended to all preceding tokens and therefore aggregates contextual information from the entire sequence:

$$pool_{last}(H) = h_T$$

This is computationally convenient since no additional aggregation step is required, and it is also the representation naturally used by the model’s output head. It performs well when the property of interest is well summarised at the sequence boundary.

Max Pooling

Max pooling takes the element-wise maximum across token positions:

$$pool_{max}(H)_j = \max_{t \in \{1, \dots, T\}} h_{t,j} \quad \text{for each dimension } j$$

This preserves the most extreme activation in each dimension and is particularly effective when the target property is triggered by specific tokens (for example, a single harmful keyword) rather than being a global property of the sequence.

Learnable Attention Pooling

Rather than using a fixed aggregation rule, attention pooling learns a weighting over token positions based on the probe’s own representation of their importance. Given a query vector

$$q \in R^d$$

the attention weights and pooled representation are:

$$\alpha_t = \frac{\exp(q^T h_t)}{\sum_{t'} \exp(q^T h_{t'})}$$

$$pool_{attn}(H) = \sum_t \alpha_t h_t$$

This allows the probe to automatically focus on the most behaviourally relevant tokens in the

sequence, rather than treating all tokens uniformly.

2.3.7 Training Procedure for Probes

Dataset Construction

We begin by assembling a labelled dataset of N input prompts, balanced across behavioural classes. For each input x_i , we:

1. Run a single forward pass of the language model on the tokenised input, extracting the hidden state matrix $H_i^{(l)}$ from a pre-selected layer l .
2. Run the model to completion using autoregressive generation, observing the output.
3. Assign a binary label

$$y_i \in \{0, 1\}$$

based on whether the observed output constitutes the target behaviour (for example, $y_i = 1$ if the model complied with a jailbreak attempt, and $y_i = 0$ if it refused).

The choice of which layer l to extract from is itself a hyperparameter. In general, intermediate layers (neither the very first nor the very last) provide the richest representations for behavioural prediction, as early layers encode primarily surface-level features while the final layers are specialised for next-token prediction.

The Role of Token Aggregation in Training

When the input consists of multiple tokens, each token’s hidden state provides a potentially informative signal. In the simplest case (last-token probing), we extract only $h_T^{(l)}$. For mean or attention pooling, we extract the full matrix $H^{(l)} \in R^{T \times d}$ and reduce it before passing it to the probe.

The appropriate aggregation strategy depends on the nature of the target property.

- If the property is localised (triggered by a specific token or short phrase) then max pooling or last-token extraction is more suitable, as mean pooling would dilute the relevant signal across the sequence.
- If instead the property reflects global context, such as the overall intent of a multi-sentence input, mean pooling or learnable attention pooling better captures the distributed signal.

2.3.8 Using Probes at Test Time

Once trained, using a probe is trivial. When a new input arrives:

1. Run the model’s forward pass on the input tokens
2. Extract the hidden state from the specified layer
3. Compute the probe’s prediction
4. This gives a probability representing the probe’s confidence that this input will trigger the predicted behavior

2.3.9 Limitations of Raw Probe Predictions

Although probes can achieve strong predictive accuracy, their outputs are not directly usable in practice. A probe might return a value such as 0.73, but it is unclear how this should be interpreted: is this high enough to justify blocking a request, or too uncertain to rely on? Neural network probabilities are often poorly calibrated, so a value labelled as “73% confidence” may correspond to a true accuracy of 60% or 85%. Without further processing, these scores provide no reliable decision threshold. This motivates the use of conformal prediction. Conformal prediction supplies a statistically grounded method for deciding when probe outputs are reliable enough to act on, by attaching explicit uncertainty guarantees. The probe gives us a signal that behaviors are predictable from hidden states; conformal prediction makes that signal actionable by adding rigorous uncertainty quantification.

2.4 Conformal Prediction

Machine learning models routinely output confidence scores, a probe might predict “jailbreak” with 73% confidence. However, these confidence scores are often misleading and poorly calibrated. A model might assign 90% confidence to predictions that are actually correct only 60% of the time, or it might be overly conservative, assigning 60% confidence to predictions that succeed 85% of the time. This miscalibration is pervasive in deep learning: neural networks learn to minimize classification errors, not to produce honest probability estimates. We need a framework that provides provable guarantees on prediction accuracy, allowing us to trust model outputs with statistical rigor.

Conformal prediction (CP) provides exactly this framework. Introduced by Vovk, Gamerman, and Shafer (2005), it is a remarkably elegant method for quantifying uncertainty that works with any pre-trained model and any data distribution. The key property that makes conformal prediction so powerful is that it is distribution-free: it makes no assumptions about whether your data is normally distributed, whether your model is well-trained, or whether your test data matches your training data perfectly. The only requirement is that your calibration data and test data are exchangeable, they come from the same process and could be shuffled together without changing their statistical properties. This is a weaker assumption than the standard “independent and identically distributed” assumption commonly made in statistics.

2.4.1 The Core Problem Conformal Prediction Solves

Imagine you have a probe that predicts jailbreak attempts. For a new input, the probe outputs a probability of 0.82. The fundamental question is: *should you trust this prediction enough to act on it?* Should you block the request based on this 82% confidence? What if the probe outputs 0.65 instead, is that still trustworthy? Without a principled threshold, you’re either too conservative (blocking many legitimate requests) or too permissive (allowing harmful requests through).

Conformal prediction solves this by automatically learning a threshold from data, with a mathematical guarantee: if you set a desired confidence level (say, 90%), then whenever the system makes a prediction, that prediction will be correct at least 90% of the time. Critically, when the system isn’t confident enough to meet this guarantee, it abstains, it defers to normal generation rather than making an unreliable prediction.

2.4.2 The Core Guarantee

Given a calibration dataset D_{cal} of n examples drawn from the same distribution as the test data, and a user-specified error rate $\alpha \in (0, 1)$ conformal prediction constructs a prediction set $C_\alpha(x)$ for each new test input x such that :

$$P(y_{test} \in C_\alpha(x_{test})) \geq 1 - \alpha$$

This is a marginal coverage guarantee: over many test inputs, the true label falls within the prediction set at least $(1 - \alpha)$ of the time.

2.4.3 How Conformal Prediction Works: A Step-by-Step Walkthrough

Let's walk through the conformal prediction procedure with a concrete example: using probes to detect jailbreak attempts.

Step 1: Start with a Trained Probe

We begin with a probe $f : R^d \rightarrow [0, 1]$ that outputs predicted probabilities. For a hidden state h , the probe produces $f(h) = [p_0, p_1]$ where p_0 is the probability the model will safely refuse the request, and p_1 is the probability it will comply (jailbreak). This probe was trained on a separate training set; conformal prediction will now *calibrate* it using fresh held-out data.

Step 2: Collect a Calibration Set

We need a calibration dataset consisting of n examples:

$$\mathcal{D}_{cal} = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$$

where each x_i is an input prompt, and $y_i \in \{0, 1\}$ is the true behavioral outcome (0 = safe refusal, 1 = jailbreak). Crucially, this data must be held out, never used for training the probe and must be exchangeable with future test data. For each calibration example, we:

- Extract the hidden state h_i from the language model
- Run the probe to get predicted probabilities $f(h_i) = [p_0^{(i)}, p_1^{(i)}]$
- Observe the true outcome y_i

Step 3: Define a Non-Conformity Score

We need a score function that measures how "unusual" it would be for example x to have label y , according to the probe. For a binary classification probe the standard non-conformity score for a candidate label is defined as

$$s(x, y) = 1 - p_\theta(x)_y$$

where $p_\theta(x)_y$ denotes the predicted probability assigned to class y . This is simply one minus the probability assigned to the true class.

Concretely:

- If the probe assigns probability 0.95 to class $y = 1$, then $s(x, 1) = 0.05$ which is low, meaning the prediction is highly conforming and hence a good prediction.
- If the probe assigns probability 0.30 to class $y = 1$, then $s(x, 1) = 0.70$ which is high, meaning the prediction is strongly non-conforming and hence a poor prediction.

The score is called a "non-conformity score" because high values indicate the example doesn't conform well to the probe's predictions.

We compute scores for all calibration examples:

$$s_i = 1 - f(h_i)_{y_i} \quad \text{for } i = 1, 2, \dots, n$$

These n scores form an empirical distribution showing how well the probe typically performs on data from this distribution.

Step 4: Calculate the Calibrated Threshold

Now comes the key step. Given a user-specified confidence level $\alpha \in (0, 1)$ (e.g., $\alpha = 0.9$ means "I want 90% confidence"), we compute a threshold as the $(1 - \alpha)$ quantile of the calibration scores:

$$\hat{q}_\alpha = \text{Quantile} \left(\{s_1, s_2, \dots, s_n\}, \frac{\lceil (n+1)(1-\alpha) \rceil}{n} \right)$$

Let's break this down:

- The quantile is a value such that a certain fraction of the data falls below it
- For $\alpha = 0.9$ and $n = 1000$, we want the $\frac{\lceil 1001 \times 0.1 \rceil}{1000} = \frac{101}{1000} = 0.101$ quantile
- This is roughly the 10th percentile, the value where 10% of calibration scores fall below

- The $\lceil (n+1)(1-\alpha) \rceil$ formula (instead of just $n(1-\alpha)$) is a small correction ensuring exact coverage guarantees

Step 5: Make Predictions with Guarantees at Test Time

When a new test input x_{test} arrives, we:

1. Extract its hidden state h_{test} and compute probe probabilities $f(h_{test}) = [p_0, p_1]$
2. Identify the predicted class: $\hat{y} = \arg \max f(h_{test})$ (the class with highest probability)
3. Compute the test non-conformity score: $s_{test} = 1 - f(h_{test})_{\hat{y}}$
4. Decision rule:
 - If $s_{test} \leq \hat{q}_\alpha$: PREDICT $\hat{y}$ (the model is confident)
 - If $s_{test} > \hat{q}_\alpha$: ABSTAIN (the model is uncertain, defer to normal generation)

The decision rule is simple: only make a prediction when the score is below the calibrated threshold (indicating the prediction is “typical” compared to calibration data). When the score exceeds the threshold, the prediction is too unusual/uncertain, so we abstain.

2.4.4 The Accuracy-Coverage Tradeoff: The Core Challenge

While conformal prediction provides guarantees, it introduces a fundamental tradeoff between two competing metrics:

- **Accuracy** (also called “conformal consistency”): Among the inputs where the system makes a prediction, what percentage are correct? For jailbreak detection, if the probe confidently predicts “jailbreak” on 100 inputs, and 92 of those actually would have triggered jailbreaks, the accuracy is 92%.
- **Coverage** (also called “selection rate”): What percentage of inputs does the system make predictions on (rather than abstaining)? If the probe is confident enough to predict on 150 out of 1000 test inputs, the coverage is 15%.

These metrics trade off against each other:

- Higher $\alpha \Rightarrow$ stricter threshold $\Rightarrow$ higher accuracy but lower coverage
- Lower $\alpha \Rightarrow$ looser threshold $\Rightarrow$ lower accuracy but higher coverage

2.5 Conformal Risk Control

Standard conformal prediction is well-suited to settings where the goal is to control miscoverage, that is, the probability that the true label falls outside the prediction set. However, many practical monitoring scenarios involve controlling a more general notion of risk, such as the expected value of an arbitrary loss function $L(x, y, \hat{y})$. Conformal Risk Control (CRC), introduced by Angelopoulos et al. (2022), extends the conformal prediction framework to this more general setting.

2.5.1 Motivation: Beyond Miscoverage

In the jailbreak detection setting, the costs of the two types of error are asymmetric. A false negative (failing to detect a jailbreak) may result in harmful content being generated, while a false positive (blocking a benign request) merely inconveniences the user. Standard conformal prediction treats both error types symmetrically via its coverage guarantee. CRC allows us to specify and control a task-specific risk function that correctly weights these asymmetric costs.

More generally, suppose we define a loss function $L(\hat{y}, y) \in [0, 1]$ that quantifies the cost of predicting $\hat{y}$ when the true label is y . Rather than simply controlling whether the true label is covered, we now want to control the *average cost* of our predictions directly:

$$E[L(\hat{y}(x_{test}), y_{test})] \leq \alpha$$

where α is the user-specified risk level. Rather than bounding the probability of a single binary event, whether or not the true label is covered, we are now bounding the expected value of an arbitrary loss function. This is strictly more general: standard conformal prediction is recovered as the special case where $L(\hat{y}, y) = \mathbf{1}[y \notin \hat{C}(x)]$, i.e. the loss is simply 1 if the true label is missing from the prediction set and 0 otherwise.

2.5.2 The CRC Framework

CRC operates by learning a threshold λ^* from the calibration data such that the resulting prediction rule satisfies the risk bound.

Let the prediction rule at threshold λ be:

$$\hat{C}_\lambda(x) = \{y : s(x, y) \leq \lambda\}$$

That is, the prediction set at threshold λ contains all labels y whose non-conformity score falls below λ . For binary classification with the standard non-conformity score, this prediction set either contains one label (confident prediction) or both labels (abstention). Given a risk function

$$R(\lambda) = \frac{1}{n} \sum_{i=1}^n L\left(\hat{C}_\lambda(x_i), y_i\right)$$

evaluated on the calibration set. CRC selects the largest λ such that the empirical risk, corrected for finite-sample uncertainty, remains within the target level α :

$$\lambda^* = \sup \left\{ \lambda \in \Lambda : \hat{R}(\lambda) + C(n, \delta) \leq \alpha \right\}$$

Choosing the *largest* valid λ is deliberate: a larger λ means more inclusive prediction sets, so we are being as permissive as possible while still respecting the risk constraint. The term $C(n, \delta)$ is a finite-sample correction that inflates the empirical risk estimate to account for the fact that $\hat{R}(\lambda)$ is computed on a finite calibration set and may underestimate the true risk.

The key requirement for this to work is that $R(\lambda)$ is monotone non-decreasing in λ . Intuitively: as λ increases, prediction sets grow larger, so the false negative rate (the fraction of true labels not in the prediction set) can only go down, never up. This means risk decreases as λ increases, satisfying the monotonicity requirement.

2.5.3 CRC for Asymmetric Safety Monitoring

In the jailbreak detection setting, the two types of classification error do not have equal consequences. A false negative, where the system fails to detect a true jailbreak attempt, may allow harmful content to be generated and is therefore significantly more costly than a false positive, where a harmless request is incorrectly flagged. For this reason, we define the risk function using the false negative rate (FNR), which measures the probability of missing a genuine jailbreak:

$$L_{FNR}(\hat{C}, y) = 1[y = 1] \cdot 1[1 \notin \hat{C}]$$

This loss is equal to 1 only when the true label is a jailbreak ($y = 1$) and the prediction set $\hat{C}$ fails to include the jailbreak label. Otherwise, the loss is 0. By controlling

$$E[L_{FNR}] \leq \alpha$$

we ensure that the system misses at most an α fraction of all true jailbreak attempts on average. CRC selects a threshold λ^* using the calibration set such that, with high probability over the calibration sample, the following bound holds:

$$FNR \leq \alpha + C(n, \delta)$$

where $C(n, \delta)$ is the finite-sample correction term that accounts for uncertainty introduced by the limited calibration data. The false positive rate, which corresponds to incorrectly flagging benign requests, is not directly constrained but is determined by the chosen threshold λ^* and can be measured empirically. This provides a principled way to control the safety-performance trade-off: we first specify the maximum acceptable miss rate for harmful requests through α , and CRC then determines the corresponding decision threshold and resulting false positive rate based on the probe’s predictive ability.

Chapter 3

Literature Review

3.1 Refusal in Language Models Is Mediated by a Single Direction

Arditi et al. [2]

This paper demonstrates that refusal behavior in language models is mediated by a one-dimensional subspace across 13 popular open-source chat models. Their core methodology involves extracting a “refusal direction” using difference-in-means between harmful and harmless instruction activations. Specifically, for each layer l and token position i , they compute:

$$r_i^{(l)} = \mu_i^{(l)} - \nu_i^{(l)} \quad (3.1)$$

where $\mu_i^{(l)}$ represents the mean activation over harmful prompts and $\nu_i^{(l)}$ represents the mean activation over harmless prompts. They then select the most effective vector r by evaluating each candidate’s ability to bypass refusal when ablated and induce refusal when added.

Their key intervention techniques include:

1. **Directional ablation** to bypass refusal by zeroing out the refusal direction component along r for every residual stream across all layers and all token positions :

$$x' \leftarrow x - \hat{r}\hat{r}^\top x \quad (3.2)$$

2. **Activation addition** to induce refusal by adding r to residual activation in the layer it

was derived from, at all token positions:

$$x^{(l)'} \leftarrow x^{(l)} + r^{(l)} \quad (3.3)$$

Critically, they show this can be implemented as a permanent weight modification through orthogonalization:

$$W'_{out} \leftarrow W_{out} - \hat{r}\hat{r}^\top W_{out} \quad (3.4)$$

This operation modifies the weight matrices to physically prevent them from ever writing the refusal direction $\hat{r}$ to the residual stream. In transformer architectures, components like attention heads and MLPs add their outputs to the residual stream via output projection matrices W_{out} . The term $\hat{r}\hat{r}^\top W_{out}$ computes the component of W_{out} that points in the refusal direction, and subtracting it removes that component entirely. This orthogonalization is applied to all matrices that write to the residual stream. The key advantage is that this is a one-time weight modification that permanently disables refusal behavior, rather than requiring repeated inference-time interventions.

This approach achieved attack success rates of up to 84% on the HarmBench dataset for certain model families while maintaining model coherence on standard benchmarks. Their mechanistic analysis of adversarial suffixes revealed that such attacks work by suppressing propagation of the refusal direction and hijacking attention of key heads away from harmful instruction content.

Relevance to current study: This work directly informs our investigation by suggesting that LLM misbehaviors may similarly be represented as linear directions in activation space. If refusal, a safety-aligned behavior, can be manipulated via a single direction, it is plausible that other misbehaviors (ignoring instructions, producing harmful content) also have corresponding directional representations. Our project could extend this methodology by: (1) extracting “misbehavior directions” using contrastive pairs of compliant vs. non-compliant outputs, and (2) testing whether adding these directions to normally well-behaved inputs can preemptively induce misbehavior, thereby validating the predictive power of linear probes before generation occurs.

3.2 The Geometry of Truth: Emergent Linear Structure in LLM Representations of True/False Datasets

Marks and Tegmark [3]

This paper investigate whether LLMs linearly represent the truth or falsehood of factual statements. Using carefully curated true/false datasets, they demonstrate that truth is encoded as a linear direction that emerges with model scale. Their probing methodology employs difference-in-means vectors:

$$\theta_{mm} = \mu^+ - \mu^- \quad (3.5)$$

where μ^+ and μ^- are the mean activations of true and false statements, respectively.

A key challenge they identify is that the simple difference-in-means direction θ_{mm} can be contaminated by other features that aren't orthogonal to the truth direction. For example, if the dataset has variation in both "truth" and "topic" (e.g., statements about different cities), and these features are correlated, the probe might pick up a mixture of both rather than pure truth. To address this, they propose mass-mean probing with inverse covariance weighting:

$$p_{mm}^{iid}(x) = \sigma(\theta_{mm}^\top \Sigma^{-1} x) \quad (3.6)$$

where Σ is the covariance matrix of the centered data (measuring how features vary together in the dataset) and σ is the logistic function. The inverse covariance factor Σ^{-1} re-scales the representation space to remove correlated noise between features, enabling the probe to isolate the truth-related direction despite overlap with other attributes.

Their key findings include:

1. PCA visualizations reveal clear linear separation between true and false statements in larger models (70B parameters)
2. Probes trained on one dataset (e.g., "cities") generalize to topically different datasets (e.g., "Spanish translations") with >95% accuracy in LLaMA-2-70B, demonstrating that the learned direction captures abstract truth rather than dataset-specific patterns.
3. Compared to logistic regression, mass-mean probes recover directions that play a stronger

causal role in the model’s predictions, attaining normalized indirect effects (NIEs) between 0.77 and 0.97 when used to intervene on truth values.. An NIE near 1.0 indicates that adding the truth direction to a false statement’s activations causes the model to predict “TRUE” with the same confidence as it would for genuinely true statements, providing strong causal evidence that this direction mediates truth representation.

They further show that truth-related directions are present even in base (non-fine-tuned) models, indicating that these representations arise during pretraining rather than from subsequent safety alignment. In addition, larger models are found to encode more abstract and transferable notions of truth that generalize across varied logical structures such as negations, conjunctions, and disjunctions. In contrast, smaller models tend to rely on lower-level cues, such as close word associations, which correlate with truth only within limited contexts.

Relevance to current study: This work establishes that behavioral features (truth/falsehood) can be linearly represented and causally manipulated in LLMs. Our project could extend this framework from truth to misbehavior prediction. Specifically, we could: (1) apply similar difference-in-means extraction to identify “instruction-following directions” or “hallucination directions” from contrastive prompt-response pairs, (2) test whether these directions transfer across different types of misbehavior (analogous to how truth directions transfer across topics), and (3) use causal intervention experiments to validate that identified directions genuinely mediate misbehavior rather than merely correlating with it. The mass-mean probing technique with covariance correction could improve our probe’s ability to isolate misbehavior-specific signals from confounding features.

Chapter 4

Language Model Behavior Prediction Framework

This section details the experimental methodology used to reproduce and extend the work of Ashok and May [1] (2025).

4.1 Problem Statement

Large language models (LLMs) are typically treated as black boxes at inference time: a prompt is submitted and the model generates a complete response before any evaluation can be performed. This is computationally expensive. For tasks where the output follows a predictable structure, such as selecting option A or B in multiple-choice questions or binary decision tasks, generating the full response may consume significantly more computation than necessary.

The work of Ashok and May (NeurIPS 2025) addresses the following fundamental question: Can a language model’s internal representations, captured during input processing and before any output is generated, reliably predict what the model will output?

If the answer is yes, then a lightweight classifier, referred to as a probe, can be trained on these internal representations to anticipate the model’s decision early. When the probe is sufficiently confident, the full generation process can be bypassed, saving substantial computational cost. This is referred to as a *behaviour anticipation* or *early-exit* system.

4.2 Methodology

1. Hidden State Extraction

For each input prompt, the LLM is run in generation mode. After the model processes the complete input sequence, but before sampling any output tokens, the hidden state vector at the final input token position is extracted from a chosen Transformer layer. Formally, let

$$h_l(x) \in R^d$$

denote the hidden state at layer l for input x , where d is the hidden dimension. The middle layer is used as the primary probe target, although all layers are evaluated in the ablation study.

2. Probe Training

A logistic regression classifier (the probe) is trained to predict a binary label y from the hidden state h :

$$P(y = 1 \mid h) = \sigma(w \cdot h + b)$$

The label y is derived from the model’s own output via a task-specific labelling function. The probe uses L_2 regularisation ($C = 1.0$) and class-balanced weights.

3. Conformal Prediction Framework

The probe is wrapped in a conformal prediction (CP) framework to provide formal accuracy guarantees. The procedure consists of four steps:

- (a) Compute non-conformity scores on the calibration set. For calibration example i :

$$s_i = 1 - \hat{P}(y_i \mid h_i)$$

- (b) Compute the conformal quantile q at confidence level α :

$$q = \text{Quantile} \left(\{s_i\}, \frac{\lceil (n+1)(1-\alpha) \rceil}{n} \right)$$

(c) At test time, predict on example j if and only if:

$$\max_c \hat{P}(c \mid h_j) \geq 1 - q$$

(d) Otherwise, abstain and allow the LLM to generate normally.

This guarantees that accuracy on predicted examples is at least α with high probability, and equivalently that the error rate is at most $1 - \alpha$.

4. The Early-Exit Mechanism

When the probe is confident (condition 3c above), the system exits early and returns the probe’s prediction directly, bypassing full LLM generation. Otherwise the LLM proceeds with normal autoregressive generation. The combined accuracy is:

$$Accuracy_{method} = \frac{\sum_{confident} 1[probe\ correct] + \sum_{uncertain} 1[LLM\ correct]}{N}$$

This can exceed raw probe accuracy, since uncertain cases fall back to the full LLM.

4.3 Dataset & Tasks

We evaluate across ten behavioural tasks spanning safety, instruction following, classification, and uncertainty quantification. For each task, a prompt is constructed from the dataset and fed to the LLM; the probe is trained to predict the LLM’s eventual output behaviour from the hidden state of the final input token alone, before any output token is generated. Table 4.1 summarises the tasks, their associated datasets, and the behaviour the probe is trained to predict.

The tasks span a broad range of difficulties and behavioural types. Classification tasks test whether the probe can anticipate the LLM’s semantic decision. Safety tasks test whether the probe can detect alignment failures before they manifest in the output. Uncertainty quantification tasks probe whether internal representations encode the model’s epistemic state, a signal invisible in the output text alone. Instruction-following tasks test format compliance prediction.

Task	Category	Datasets	Probe predicts
MCQA	Classification	MMLU, ARC, PIQA, HellaSwag, MedMCQA, CommonsenseQA, OpenBookQA, QASC, CosmoQA	Chosen option (A or B)
Sentiment	Classification	IMDB, Yelp Polarity, Amazon-Reviews, MTEB Twitter, TwitterFinance, NewsMTC, Financial Phrasebank, AuditorSentiment, Emotion, SST-5	Positive or Negative
Fact Verification	Classification	FEVER, HealthVER, Climate-FEVER	True or False
News Topic	Classification	AGNews, BBCNews, NYTimes	Predicted topic label
Toxicity	Classification	JigsawToxicity, JigsawUnintendedBias	Toxic or Safe
Unanswerable	Safety / Alignment	SelfAware, KnownUnknown	Unanswerable or not
Jailbreak	Safety / Alignment	WildJailbreak	Abstain or comply
Confidence	Uncertainty Quantification	NaturalQA, MSMarco, TriviaQA	High or low perplexity output
Verbal Confidence	Uncertainty Quantification	NaturalQA, MSMarco, TriviaQA	Confident or Uncertain
Format: Bullets	Instruction Following	NaturalQA, MSMarco, TriviaQA	Correct 3-bullet format or not
Format: JSON	Instruction Following	NaturalQA, MSMarco, TriviaQA	Valid JSON output or not
Random Token	Control	NaturalQA, MSMarco, TriviaQA	Random (A) or (B) token

Table 4.1: Tasks, datasets, and probe prediction targets.

4.4 Pipeline

Stage 1 — Data Preparation (`data.py`)

Downloads a raw dataset from HuggingFace, applies task-specific preprocessing, and formats each example into a prompt containing: (1) a system instruction defining the task format, (2) two few-shot worked examples separated by [STOP] tokens, and (3) the question or text to classify followed by an incomplete response stem. The output is a pair of train/test CSV files with `text`, `idx`, and optional `label` columns.

Stage 2 — LLM Inference (`save_inference.py`)

Runs the LLM on every prompt and captures three outputs per example: (1) generated output text (truncated at [STOP] or `max_new_tokens`) (2) hidden state vectors from the middle transformer layer (primary) and every 4th layer from layer 1 to the last layer, each extracted from the last input token at the first generation step and (3) perplexity computed via `model.compute_transition_scores()`.

Stage 3 — Labelling (`label.py`)

Parses the LLM’s raw output into a numeric behavioural label reflecting what the model actually did. Unparseable outputs are assigned `None` and removed before probe training. The label is written to the CSV as a `model_label` column.

Stage 4 — Probe Training (`modeling.py`)

Loads hidden states and labels, then implements the probe and conformal framework described:

1. Align hidden states and CSV, truncating the CSV if lengths differ; drop null-labelled examples.
2. Split training data 85/15 into probe training and conformal calibration sets.
3. Fit the logistic regression probe on `probe_train`.
4. Compute `predict_proba()` on calibration and test sets.
5. Compute conformal quantile q and apply threshold $1 - q$ to the test set.

Stage 5 — Evaluation and Analysis (`ablations.py`, `analyze_ablations.py`)

Performs systematic ablations and computes primary and secondary metrics.

- **Primary metrics:** conformal accuracy, coverage, raw probe accuracy, and base rate.
- **Secondary metrics:** token savings, correlation of probe correctness with model correctness, correlation with input/output length, and probe confidence vs. perplexity correlation.
- **Ablation dimensions:** data scale (1% to 100%), layer index, probe architecture, and confidence level α .

Chapter 5

Own Implementation

5.1 Tech Stack

The implementation runs as a single Google Colab notebook that orchestrates the paper’s own repository code as subprocesses alongside custom Python logic for the extended methods. The following table describes each component and the justification for its selection.

Component	Role and Justification
meta-llama/Llama-3.2-3B-Instruct	Language model used for inference. Chosen because it is small enough to run on a single Colab T4 GPU (16 GB VRAM) while still being a genuine instruction-tuned transformer. It has 28 layers and a 3072-dimensional hidden state, making it suitable for probing experiments. The paper evaluated across five models, Llama 8B, 3B, 70B, DeepSeek-R1-Distill-Qwen-14B, and Mistral-7B-Instruct, and found that probe performance scales favourably with model size. Llama-3.2-3B-Instruct sits at the smaller end of this range, matching one of the paper’s own experimental models, making results directly comparable rather than being an approximation.
HuggingFace Transformers	Provides <code>AutoModelForCausalLM</code> , <code>AutoTokenizer</code> , and <code>AutoConfig</code> . Used to load Llama-3.2-3B-Instruct and run <code>generate()</code> with <code>output_hidden_states=True</code> . This is the same library used by the paper’s <code>save_inference.py</code> .

Component	Role and Justification
HuggingFace Datasets	Used to load the SelfAware dataset (<code>JesusCrist/selfAware</code>) from the HuggingFace Hub. The full dataset is loaded into memory via <code>load_dataset()</code> and then sampled down to the required number of examples.
NumPy	Core array library for all hidden state operations. The <code>.npy</code> format used by <code>save_inference.py</code> for storing hidden states is a NumPy binary format. All conformal calibration arithmetic is implemented in NumPy.
scikit-learn	Provides <code>LogisticRegression</code> (the probe), <code>StandardScaler</code> (feature normalisation), <code>train_test_split</code> (probe/calibration split), and all evaluation metrics (<code>accuracy_score</code> , <code>confusion_matrix</code> , <code>classification_report</code>).
pandas	Used to load and manipulate the inference CSVs produced by <code>save_inference.py</code> and <code>label.py</code> .
matplotlib + seaborn	Used for the paper-style visualisations.
Paper repo (subprocess)	The paper’s own <code>save_inference.py</code> , <code>label.py</code> , <code>models.py</code> , <code>modeling.py</code> , and <code>ablations.py</code> are cloned from GitHub and called as subprocesses or imported. This ensures fidelity to the paper’s pipeline rather than reimplementing it independently.
Google Colab (T4 GPU)	Runtime environment. The T4 has 16 GB VRAM, sufficient for Llama-3.2-3B-Instruct inference with bfloat16 weights.

Table 5.1: Implementation tech stack and justification

Compatibility Patches

Two patches are applied to the paper’s `save_inference.py` before use to resolve compatibility issues with Llama-3.2-3B-Instruct.

- **Patch 1 - BFloat16**

- Llama-3.2-3B-Instruct stores activations as `bfloat16` tensors which NumPy cannot direct convert
- Every `.detach().cpu().numpy()` call is replaced with: `.detach().float().cpu().numpy()`

- This casts tensors to `float32` before conversion and prevents runtime failures during hidden state saving.

- **Patch 2 - Attentions**

- Llama-3.2-3B-Instruct uses PyTorch’s Scaled Dot-Product Attention (SDPA) fused kernel.
- This kernel does not support `output_attentions=True`.
- Lines attempting to access `output.attentions` are commented out.
- This does not affect the experiment since only hidden states are required, and they remain fully supported.

5.2 Code Walkthrough

1. Installs and Imports

Loads all required libraries and verifies GPU availability.

- Imports scikit-learn, NumPy, pandas, matplotlib, seaborn, and PyTorch.
- Clones the paper’s GitHub repository so `save_inference.py` and `label.py` can be called as subprocesses.
- Applies two compatibility patches to `save_inference.py`: a BFloat16 cast fix (Llama uses bfloat16, which NumPy cannot convert directly) and an attentions fix (Llama’s SDPA attention does not support `output_attentions=True`). Logs in to Hugging-Face.

2. Configuration

Centralises all hyperparameters and path definitions.

- `PROBE_LAYER = 14`: the midpoint of Llama-3.2-3B’s 28 transformer layers, matching `save_inference.py`’s default.
- `TRAIN_STOP_IDX = 600`, `TEST_STOP_IDX = 300`: caps on examples processed, sized to fit within Colab session limits.
- `ALPHA_VALUES = [0.6, 0.7, 0.8, 0.91, 0.95, 0.99]`: sweeps from permissive to strict, with 0.91 matching the paper’s default.

- Sets environment variables read internally by `modeling.py` and `ablations.py` via `os.getenv()`.

3. Dataset Construction

Creates train and test prompt CSVs from the SELF-AWARE dataset.

- Downloads SELF-AWARE from HuggingFace and filters questions between 10 and 500 characters, excluding vague “would you rather” questions.
- Constructs prompts using the paper’s exact few-shot format from `data.py`: a system instruction followed by four worked examples showing the expected **Thinking:** ... **Answer:** ... format, ending with **Thinking:** for Llama to complete.
- Balances classes by taking equal numbers of answerable and unanswerable examples, then shuffles and splits **60/40** into train and test sets. The `label` column records ground truth answerability; the probe target `model_label` is created in Block 5 after Llama-3.2-3B has run.

4. Run `save_inference.py`

Calls the paper’s inference script as a subprocess on both the train and test sets using `meta-llama/Llama-3.2-3B-Instruct`. For each prompt, the script runs `model.generate()` with hidden states enabled, extracts the hidden state at the last input token for the primary layer and every fourth layer, and saves outputs to `.npy` files and enriched CSVs (output and `perplexity` columns). CSVs are trimmed to exactly `stop_idx` rows after both calls. Results are saved to Google Drive to persist across Colab sessions.

5. Run `label.py`

Parses Llama’s raw output into a numeric `model_label`. Under the `--config unanswerable` setting, `model_label = 1` if the output contains the token “Unanswerable”, and 0 otherwise. A patch is applied beforehand to guard against NaN/None outputs from failed generations. The raw `model_label` is then overwritten with a correctness label:

Label 1 = Model’s response matches ground truth
 Label 0 = Model’s response does not match ground truth

Under this correctness label, 1 means the model responded correctly (e.g., refusing an unanswerable question or answering an answerable one appropriately), and 0 means it misbehaved (e.g., hallucinating an answer or refusing a valid question). Thus, the final

label aligns directly with the safety goal: label 1 can pass through safely, and label 0 should be blocked or warned against.

6. Load Hidden States and Labels

Loads artefacts from Blocks 4 and 5 into memory, replicating `get_xydf()` from `modeling.py`.

- Reads the inference CSV and loads `hidden_states_{layer}.numpy` $\rightarrow$ shape $(N, 2048)$ for Llama-3.2-3B-Instruct.
- Trims the dataframe to `len(X)` if lengths differ, then drops rows where `model_label` is null, applying the same boolean mask to both `X` and the dataframe.

7. Pre-process Hidden States

Standardises feature vectors to zero mean and unit variance, mirroring the `StandardScaler` in `models.py`. The scaler is fit on training data only; the same parameters are applied to the test set without refitting.

Hidden state dimensions vary enormously in magnitude across a 2048-dimensional vector. Standardisation ensures each dimension contributes comparably to the decision boundary, improving convergence speed and final accuracy.

8. Train/Calibration Split

Partitions training data into `probe_train` (85%) and calibration (15%), mirroring `validation_split=0.15` in `modeling.py`. Stratification on `y_train` ensures class balance in both splits matches the full training set, because the conformal quantile calculation assumes calibration scores are representative of the test distribution.

9. Train the Linear Probe

Fits logistic regression on `probe_train`, mirroring the `Linear` class in `models.py`. `predict_proba()` is called on the calibration and test sets, producing $(N, 2)$ probability matrices used by all downstream methods.

10. Conformal Calibration

For each α value, computes the conformal quantile q and builds prediction sets, matching `metrics.py` exactly.

- `calibrate_conformal()`: computes non-conformity scores $s = 1 - \hat{P}(y_{true})$ on the calibration set and returns the finite-sample corrected quantile q .

- `build_prediction_set()`: includes class c in the prediction set if $\hat{P}(c) \geq 1 - q$, yielding set types $\{\}$, $\{0\}$, $\{1\}$, $\{0,1\}$.
- Results are cached in `calibration_cache[alpha]` to avoid recomputation.

11. Methods 1–5

Each block implements a distinct decision strategy on top of the probe probabilities. These are discussed in detail in Section 5.3.

5.3 Methods and Design Decisions

Five distinct decision strategies are evaluated. Method 1 is the baseline. Method 2 reproduces the paper’s approach. Methods 3–5 are original extensions.

5.3.1 Method 1 : Baseline (Raw Probe)

The baseline applies the probe to every test example and always returns the highest-probability class with no confidence filtering whatsoever:

$$\hat{y}_i = \arg \max_c \hat{P}(c \mid h_i) \quad \text{for all } i$$

Because no example is ever abstained on, the abstention rate is 0% by definition. This makes it the upper bound on coverage (every example is predicted) and the lower bound on accuracy (no filtering is applied to remove uncertain predictions).

The baseline is essential as a reference point. All conformal methods below trade coverage (predicting on fewer examples) for accuracy (being more reliable on those they do predict). Comparing any method against the baseline isolates the accuracy gain attributable purely to abstention.

5.3.2 Method 2 : Original CP Threshold (Paper’s Approach)

This is the paper’s core method. Rather than always predicting, the probe only commits to a prediction when it is sufficiently confident. The confidence threshold is not chosen arbitrarily — it is derived from the calibration set using conformal prediction, which provides a formal statistical guarantee on accuracy.

The procedure is as follows:

1. Compute a non-conformity score for each calibration example i , measuring how surprised the probe is by the true label:

$$s_i = 1 - \hat{P}(y_i \mid h_i)$$

A low score means the probe was confident and correct. A high score means the probe assigned low probability to the true class.

2. Compute the conformal quantile q from these scores at the desired confidence level α :

$$q = \text{Quantile}\left(\{s_i\}, \frac{\lceil (n+1)(1-\alpha) \rceil}{n}\right)$$

The finite-sample correction $\lceil (n+1)(1-\alpha) \rceil/n$ (rather than $1-\alpha$ directly) ensures the guarantee holds exactly for finite calibration sets, not just asymptotically.

3. At test time, the probe predicts on example j if and only if its maximum predicted probability clears the threshold derived from q :

$$\max_c \hat{P}(c \mid h_j) \geq 1 - q$$

If this condition holds, return $\max_c \hat{P}(c \mid h_j)$. If not, fall back to the LLM’s actual output (`model_label`).

Implementation note on fallback labels :

The paper’s fallback rule requires knowledge of what the LLM actually output on abstained examples, concretely, the `model_label` column as produced by `label.py`, where 1 denotes that the LLM said “Unanswerable” and 0 denotes that it gave an answer.

In our implementation, this column was subsequently overwritten in preprocessing (Block 5B) to instead encode correctness: 1 if the LLM’s response matched the ground truth, 0 otherwise. By the time Method 2 is evaluated, the original per-example LLM output is no longer available in memory. As a consequence, the fallback cannot use the true `model_label` and instead defaults to `POSITIVE_LABEL=1` (safe/correct) for all abstained examples. This means abstained examples are assumed to be correct LLM responses rather than reflecting what the model actually did.

Two accuracy values are reported:

- **Conformal accuracy:** accuracy only on the confident subset. This is the value the CP framework guarantees to be $\geq \alpha$ with high probability. Since conformal accuracy is computed exclusively on the confident subset, it is unaffected by the fallback limitation described above and is the primary metric used in all headline comparisons.
- **Method accuracy:** accuracy across all N test examples, combining probe predictions on confident cases and LLM fallbacks on uncertain ones. This can exceed raw probe

accuracy because uncertain examples are handled by the full LLM, which may still be correct. In our implementation, abstained examples are assigned a constant fallback of `POSITIVE_LABEL` rather than the LLM’s true output (which was overwritten during preprocessing), so this value is artificially inflated and is tracked internally but excluded from the results table.

5.3.3 Method 3 : Singleton Rule with Prediction Sets

Method 3 extends the conformal framework from a scalar threshold to full *prediction sets*. Instead of asking “is the probe’s top probability high enough?”, it asks “which classes are individually plausible given the calibration threshold?”. For each test example j , the prediction set is:

$$S_j = \left\{ c : \hat{P}(c \mid h_j) \geq 1 - q \right\}$$

In binary classification, four outcomes are possible:

(label 1 = model behaves; label 0 = model misbehaves):

1. $\mathbf{S_j = \{1\}}$: only class 1 clears the threshold. The probe is confident the model will respond correctly $\Rightarrow$ Pass through
2. $\mathbf{S_j = \{0\}}$: only class 0 clears the threshold. The probe is confident the model will misbehave $\Rightarrow$ Block
3. $\mathbf{S_j = \{0, 1\}}$: both classes clear the threshold. The probe is uncertain — both outcomes are plausible.
4. $\mathbf{S_j = \{\}}$: neither class clears the threshold. The probe cannot support either prediction, typically signalling an out-of-distribution example.

Method 2 cannot distinguish between them because it only observes the scalar maximum probability. Method 3 makes this distinction explicit and offers two strategies for handling the non-singleton cases:

- **Method 3a — Singleton + Conservative Rule:** when S_j is ambiguous or empty, default to predicting misbehave (0). This is a safety-first heuristic: in behaviour anticipation, a false negative (failing to flag a misbehaviour) is more costly than a false positive. Defaulting to 0 under uncertainty minimises the missed-misbehaviour rate at the cost of more false alarms.

- **Method 3b — Singleton + Abstain Rule** when S_j is not a singleton, abstain entirely and fall back to the LLM. This prioritises precision, the probe only commits when it is unambiguously confident in exactly one class. This produces higher accuracy on predicted examples than 3a, but at the cost of a higher abstention rate.

Issue with proposed Method 3b:

The initial design of Method 3b used the same shared threshold q as Method 3a, the single quantile computed from all calibration examples. This creates a fundamental problem. Because $P(0) + P(1) = 1$, whenever $t > 0.5$, at most one class can ever clear the threshold. The ambiguous case $\{0, 1\}$ is therefore *structurally impossible*: both classes cannot simultaneously satisfy $P(c) \geq t$ when their probabilities sum to one. This means Method 2 and the original Method 3b always produce identical decisions.

Consider a shared threshold $t = 0.7$:

Method 2 predict if $\max(P_0, P_1) \geq 0.7$, then predict argmax

Current 3b include class c in set if $P(c) \geq 0.7$

If $P(1) = 0.8$, then $P(0) = 0.2$, so only $P(1)$ clears 0.7, so the set is $\{1\}$ a singleton and so both methods predict 1. They agree on every single example. The prediction set structure adds no new information.

The fix: Class-Conditional Singleton + Abstain Rule

The revised Method 3b calibrates a *separate* threshold for each class, fitted independently on calibration examples belonging to that class only:

$$t(c) = 1 - \hat{q}_c$$

where $\hat{q}_c$ denotes the conformal quantile estimated from calibration examples belonging to class c . Because $t(0)$ and $t(1)$ are now decoupled, both $P(0) \geq t(0)$ and $P(1) \geq t(1)$ can hold simultaneously, making $\{0, 1\}$ possible.

Consider $t(0) = 0.4$ and $t(1) = 0.6$.

Example	Calculation	Prediction set
$P(0) = 0.45, P(1) = 0.55$	$0.45 \geq 0.4 \quad 0.55 \geq 0.6 \times$	$\{0\} \Rightarrow \text{Block}$
$P(0) = 0.45, P(1) = 0.65$	$0.45 \geq 0.4 \quad 0.65 \geq 0.6$	$\{0, 1\} \Rightarrow \text{Abstain}$

The second case was impossible under a shared threshold. The probe can explicitly recognise borderline inputs and consciously choose not to intervene, rather than forcing a guess it is likely to get wrong. This is what makes the revised Method 3b meaningfully different from both Method 2 and the original Method 3b.

5.3.4 Method 4 : Conformal Risk Control (CRC)

Methods 2 and 3 control overall accuracy on the predicted subset. Method 4 targets a different quantity: the *false negative rate* (FNR), the fraction of true misbehaviours (class 0) that the probe misses. In safety-critical settings, this is arguably the more important metric, since failing to anticipate a misbehaviour is more dangerous than a false alarm.

Assuming label 1 = model behaves; label 0 = model misbehaves, the calibration procedure works as follows:

1. Fix a target FNR τ (values tested: $\tau \in \{0.05, 0.10, 0.15, 0.20, 0.30, 0.40\}$).
2. Sweep confidence thresholds t from high to low. For each t , compute the empirical FNR on the calibration set:

$$FNR_{calib}(t) = \frac{\#\{i \in calib : y_i = 0 \text{ and } \arg \max_c \hat{P}(c | h_i) \neq 0\}}{\#\{i \in calib : y_i = 0\}}$$

3. Select the lowest threshold t^* under which the calibration FNR does not exceed τ :

$$t^* = \min \{ t : FNR_{calib}(t) \leq \tau \}$$

4. At test time, predict on example j if and only if $\max_c \hat{P}(c | h_j) \geq t^*$. Otherwise fall back to the LLM.

A lower τ yields a stricter threshold: fewer predictions are made but missed misbehaviours are rare. A higher τ recovers more coverage at the cost of a higher missed-misbehaviour rate. This is an instance of conformal risk control applied to the FNR risk function rather than the standard error rate.

5.3.5 Method 5 : Misbehaviour Direction + CP Rule

Methods 1–4 all train a logistic regression probe via gradient optimisation. Method 5 replaces this with a geometry-based, parameter-free classifier. The core intuition is that safe and mis-

behave examples occupy distinct regions of hidden-state space, and the vector connecting their class means is sufficient for classification.

The procedure is as follows.

1. Compute the misbehaviour direction.

Using the probe training set, compute the mean hidden-state vector for each class:

$$\mu_{misbehave} = \frac{1}{n_0} \sum_{i: y_i=0} h_i, \quad \mu_{safe} = \frac{1}{n_1} \sum_{i: y_i=1} h_i.$$

The misbehaviour direction is the vector pointing *from* the safe mean *towards* the misbehave mean:

$$d = \mu_{misbehave} - \mu_{safe}.$$

Normalise to unit length: $\hat{d} = d / \|d\|$.

2. Project every example onto the direction.

For each hidden state h_i , compute a scalar projection score:

$$s_i = h_i \cdot \hat{d} \in \mathbb{R}.$$

A high score indicates the hidden state points strongly towards the misbehave mean; a low score indicates it points towards the safe mean.

3. Calibrate class-conditional CP thresholds.

Recall that a high projection score s_i means the example looks like misbehave, and a low score means it looks like safe. We want a *nonconformity score* for each class, a number that is low when an example fits that class well, and high when it does not. The two classes require opposite conventions:

- **Misbehave class (label 0):** a high s_i means the example fits the misbehave class well, so nonconformity should be low. We therefore flip the sign:

$$A_i^{(0)} = -s_i.$$

A strongly misbehave-like example has a large positive s_i , giving a large negative $A_i^{(0)}$, a very low nonconformity.

- **Safe class (label 1):** a low s_i means the example fits the safe class well, so nonconformity should also be low when s_i is low. We keep the sign unchanged:

$$A_i^{(1)} = +s_i.$$

A strongly safe-like example has a small (or negative) s_i , giving a small $A_i^{(1)}$, a low nonconformity.

Once we have the nonconformity scores, we calibrate a threshold q_c for each class separately. Intuitively, q_c answers the question: *“how nonconforming can an example be and still plausibly belong to class c ?”*

The threshold is set at a high quantile of the calibration nonconformity scores for class c :

$$q_c = \text{Quantile}\left(\{A_i^{(c)} : y_i = c\}, \frac{\lceil (n_c + 1)(1 - \alpha) \rceil}{n_c}\right).$$

Concretely: if $\alpha = 0.1$, the threshold q_c is set so that 90% of the calibration examples from class c have nonconformity $\leq q_c$. Any test example with nonconformity above q_c is considered too dissimilar from class c to be included in the prediction set for that class.

4. Build a prediction set for each test example.

Example i is assigned the set of classes whose nonconformity is within the calibrated threshold:

$$C_i = \{c \in \{0, 1\} : A_i^{(c)} \leq q_c\}.$$

This yields one of four possible sets: $\emptyset$, $\{0\}$, $\{1\}$, or $\{0, 1\}$.

5. Apply the conservative decision rule.

The same rule as Method 3a is used:

$$\hat{y}_i = \begin{cases} 1 & \text{if } C_i = \{1\} \\ 0 & \text{otherwise (i.e. } C_i \in \{\{0\}, \{0, 1\}, \emptyset\}) \end{cases}$$

Ambiguous ($\{0, 1\}$) and empty ($\emptyset$) sets are both resolved as misbehave, reflecting a safety-motivated preference for low false-negative rate.

Method 5 never abstains; every test example receives a prediction. The key advantage over logistic regression is that d is computed in closed form from the class means alone, requiring no

optimisation. This makes Method 5 more data-efficient in the small-sample regime and more interpretable, since the direction has a direct geometric meaning in hidden-state space.

Chapter 6

Results

This section presents results from the Unanswerable Question Detection experiment using Llama-3.2-3B-Instruct on the SelfAware dataset. Prior work has shown that internal activations, recorded before any output is generated, contain useful signals about a model’s future behaviour. Building on this, the central research question is whether these signals can be leveraged to provide reliable, actionable warnings for a substantial fraction of inputs while still detecting most critical failures.

6.1 Experimental Setup

The experiment uses Llama-3.2-3B-Instruct (28 transformer layers, 3072-dimensional hidden states), probed at layer 14 (the middle layer) with 600 training examples and 300 test examples. The label convention is importantly different from a naive implementation:

- $\text{model_label} = 1 \Rightarrow$ Llama’s response MATCHES the ground truth
 - Correctly identifies an unanswerable question as unanswerable or correctly answers an answerable question
- $\text{model_label} = 0 \Rightarrow$ Llama’s response DOESN’T MATCH the ground truth
 - The Model made an error
 - Hallucinates an answer to an unanswerable question or refuses to answer a question it should have answered

Justification for labeling:

The label is not “*Did the model say Unanswerable?*” but “*Did the model make the right call?*”. This distinction is important. The raw output token and the correctness label can disagree in two ways:

Case 1 : Hallucinated abstention.

The question has a clear factual answer, but the model outputs "Unanswerable". The keyword label gives $model_label = 1$ (produced the target token). The correctness label gives $model_label = 0$ as the model made the wrong call, it should have answered.

Case 2 : Hallucinated answer.

The question is genuinely unanswerable, but the model confidently produces an answer anyway. The keyword label gives $model_label = 0$ (did not produce target token). The correctness label also gives $model_label = 0$, but for the right reason, the model failed to recognize its own knowledge limit.

By training on correctness labels, the probe is forced to learn something deeper than surface output patterns. It must predict, from the model’s internal hidden states alone and before any output is generated, whether the model’s self-assessment of the question is going to be right or wrong. This is a strictly harder task than predicting output tokens, and it is the task that actually matters for deployment: a safety system does not need to know what the model will say, it needs to know whether the model can be trusted on this particular input.

Metric	Value
Model	Llama-3.2-3B-Instruct
Probe Layer	14 (middle of 28)
Hidden State Dimension	3072
Train & Test Size	Train = 600 & Test = 300
Train Labels	Label 0 = 198 & Label 1 = 402
Test Labels	Label 0 = 101 & Label 1 = 199
Train & Calibration Split (85/15)	Probe Train = 510 & CP Calibration = 90
Base rate (majority class)	66.33%
Baseline accuracy (Raw Probe)	64.67%
Baseline failure miss rate (Raw Probe)	57.43%

Table 6.1: Experimental configuration and dataset statistics for the Unanswerable Question Detection task.

6.2 Method 1 : Baseline Probe Performance

The raw logistic regression probe achieves 64.67% accuracy, slightly below the 66.33% majority-class baseline. At first glance this appears poor, but raw accuracy is misleading in this setting. The probe is trained with `class_weight='balanced'`, which deliberately sacrifices overall accuracy to improve detection of the minority class (label 0, the misbehaviour class). Without this adjustment, the model could achieve around 66% accuracy simply by predicting label 1 for every example, correctly classifying most cases while completely failing to detect true failures, which is not ideal in a safety-critical setting.

A more meaningful metric is therefore the failure miss rate (a true misbehave example being classified as safe). The baseline probe misses 57.43% of true failures, failing to identify 58 out of 101 misbehaviour cases. This establishes the starting point that all later methods aim to improve. Importantly, although raw accuracy is slightly lower, the probe still outperforms majority-class prediction on balanced metrics, achieving a macro F1 score of 0.59 compared to 0.50 for always predicting the majority class.

```
Accuracy: 0.6467
Failure miss rate : 57.4%
Failure miss rate = percentage of true label 0 cases that the probe incorrectly predicted as label 1
```

	precision	recall	f1-score	support
Incorrect limit awareness (0)	0.47	0.43	0.45	101
Correct limit awareness (1)	0.72	0.76	0.74	199
accuracy			0.65	300
macro avg	0.60	0.59	0.59	300
weighted avg	0.64	0.65	0.64	300

```
Base rate (majority class %): 66.33%
```

Figure 6.1: Baseline performance of the raw logistic regression probe

6.3 Method 2 : CP Threshold (Paper’s Core Method)

Method 2 is the central contribution from Ashok and May [1] (2025). The idea is that by abstaining on uncertain examples, the probe can trade coverage for accuracy on the examples it does act on. The conformal framework provides a statistical guarantee: accuracy on predicted examples should be at least α .

Alpha	0.60	0.70	0.80	0.91	0.95	0.99
Threshold	0.9698	0.9948	0.9983	0.9999	1.0000	1.0000
Conformal Accuracy	72.19%	76.52%	77.45%	80.00%	74.19%	100.00%
Coverage	62.33%	44.00%	34.00%	18.33%	10.33%	0.33%
Failure Miss Rate	53.45%	48.57%	48.28%	43.75%	50.00%	0.00%
False Alarm Rate	16.28%	14.43%	12.33%	10.26%	14.29%	0.00%
Predicted Samples	187/300	132/300	102/300	55/300	31/300	1/300

Table 6.2: Method 2 (Paper CP Threshold) results across different confidence levels α .

The conformal accuracy reaches 80.00% at $\alpha = 0.91$, clearly above the base rate of 66.33%, showing that the probe is genuinely discriminative when it is sufficiently confident. Coverage at this level is 18.33%, meaning the probe produces predictions for only 55 out of 300 test examples.

An interesting result is that accuracy drops from 77.45% at $\alpha = 0.80$ to 74.19% at $\alpha = 0.95$, before increasing sharply to 100.00% at $\alpha = 0.99$. The accuracy non-monotonicity between $\alpha = 0.80$ and $\alpha = 0.95$ is most likely a calibration artefact caused by the small calibration set. With only 90 calibration examples, the empirical quantile is estimated from a limited sample. The finite-sample correction formula,

$$\lceil (n+1)(1-\alpha) \rceil / n$$

becomes particularly unstable at high confidence levels. For example, with $n = 90$ and $\alpha = 0.95$, the corrected level becomes

$$\lceil 91 \times 0.05 \rceil / 90 = 5/90 \approx 0.056$$

meaning the threshold is determined by approximately the 5.6th percentile of the nonconformity

scores. At this level, even a single outlier among the 90 calibration scores can shift the threshold significantly, causing the selected examples at $\alpha = 0.95$ to differ substantially from those at $\alpha = 0.91$. As a result, the probe may retain a smaller but accidentally less reliable subset, producing the observed drop in accuracy.

The failure miss rate tells a more concerning story. Even at $\alpha = 0.91$, where conformal accuracy is relatively high, the failure miss rate remains 43.75%, meaning nearly half of the true misbehaviours within the selected set are still missed. To understand why, it is important to distinguish what conformal accuracy actually guarantees. The CP framework guarantees that across all examples the probe commits to, the fraction it gets right is $\geq \alpha$. However, this guarantee is marginal, it holds over the full confident subset, not per class. This means a probe can achieve 91% conformal accuracy while being systematically wrong on one class, provided that class is rare enough that its errors do not drag the overall accuracy below α .

In the SelfAware task, misbehaviours (label 0) are the minority class by construction, the dataset is filtered to keep only cases where the LLM failed to abstain, which are less frequent than correct abstentions. The probe therefore encounters far more label-1 examples during calibration, and the threshold q is set to satisfy the marginal guarantee over this imbalanced mixture. A probe that is highly confident in predicting label-1 on nearly every example can clear the α bar while missing the majority of label-0 cases entirely. High confidence does not guarantee confidence in the correct direction: the probe may assign high probability to label 1 precisely on the examples where the true label is 0, because misbehaviour cases may look superficially similar to correct ones in the hidden state space. The result is that high selective accuracy and high false negative rate can coexist whenever the probe’s errors are concentrated on the minority class. This is particularly problematic for safety-critical settings, where missing a failure is typically far more costly than raising a false alarm.

6.4 Method 3a : Singleton + Conservative Rule

Method 3a is designed to address the main weakness of Method 2: abstention. By never abstaining and instead flagging any uncertain example as misbehaviour, it guarantees 100% coverage. The rationale is straightforward: in a safety-critical system, the cost of a missed misbehaviour far outweighs the cost of an unnecessary warning.

Alpha	0.60	0.70	0.80	0.91	0.95	0.99
Accuracy	59.33%	55.67%	50.33%	43.00%	38.00%	34.00%
Coverage	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
Failure Miss Rate	30.69%	16.83%	13.86%	6.93%	4.95%	0.00%
False Alarm Rate	45.73%	58.29%	67.84%	82.41%	90.95%	99.50%
Conservative Predictions	113/300	168/300	198/300	245/300	269/300	299/300

Table 6.3: Method 3a (Singleton + Conservative Rule) results across different confidence levels α

The failure miss rate decreases dramatically as α increases, falling from 30.69% at $\alpha = 0.60$ to 0.00% at $\alpha = 0.99$. This is exactly the behaviour the conservative classification rule is designed to achieve. At $\alpha = 0.91$, only 7 true misbehaviours are missed (6.93% FNR), compared to 58 missed by the raw probe baseline, representing a substantial improvement in failure detection.

However, this improvement comes at a severe cost. At $\alpha = 0.91$, the false alarm rate rises to 82.41%, meaning the probe incorrectly flags 164 of the 199 safe (correct) examples as misbehaviour. In other words, operators would receive warnings for more than four out of every five correct model responses. Although this method successfully reduces missed failures, such an extreme false alarm rate would make the system highly impractical for real-world deployment, as constant false warnings would quickly reduce trust in the system and create alert fatigue.

An apparent paradox emerges in the overall accuracy results: Method 3a’s accuracy decreases steadily as α increases, dropping from 59.33% to 34.00%. At first glance, this seems counterintuitive, since stricter thresholds might be expected to improve performance. However, accuracy here is measured across all 300 test examples rather than only the risky subset. As α increases, more examples are conservatively classified as misbehaviour (label 0). Since 199 out of 300 examples (66.33%) actually belong to label 1 (safe behaviour), aggressively predicting label 0 causes many correct responses to be incorrectly flagged, which lowers overall accuracy.

Therefore, the method is not becoming worse in the safety sense; rather, it is explicitly trading general accuracy for stronger failure prevention. This reflects the central objective of conserva-

tive classification: prioritising the reduction of false negatives (missed failures) even when this substantially increases false positives (false alarms).

6.5 Method 3b : Class-Conditional Singleton + Abstain Rule

Method 3b is a meaningful technical departure from both Methods 2 and 3a. It uses separate calibration thresholds for each class, rather than a shared threshold derived from all calibration examples. This is important because the two classes may have very different confidence profiles: the probe might be highly confident about one class and uncertain about the other.

The key consequence of class-conditional calibration is that 0,1 and sets can now occur naturally, because class 0's threshold and class 1's threshold are independent. In the shared-threshold version (Methods 2 and 3a), both classes use the same $1-q$ cutoff, making it mathematically impossible for both to simultaneously clear the threshold (since $P_0 + P_1 = 1$ in binary classification with a softmax). Under class-conditional calibration, the two thresholds are calibrated from different subsets, so they don't sum to 1 and genuine ambiguity becomes possible.

Alpha	0.60	0.70	0.80	0.91	0.95	0.99
$t(0)$	0.7745	0.9268	0.9663	0.9962	0.9995	0.9999
$t(1)$	0.9944	0.9973	0.9988	1.0000	1.0000	1.0000
Accuracy	69.64%	71.53%	71.31%	67.24%	62.86%	71.43%
Coverage	56.00%	48.00%	40.67%	19.33%	11.67%	4.67%
Miss Rate	34.62%	32.61%	32.50%	23.81%	28.57%	0.00%
False Alarm	28.45%	26.53%	26.83%	37.84%	42.86%	80.00%

Table 6.4: Method 3b (Class-Conditional Singleton + Abstain) performance metrics across α .

Alpha	0.60	0.70	0.80	0.91	0.95	0.99
Singleton 0	67	57	49	30	19	13
Singleton 1	101	87	73	28	16	1
Both	0	0	0	0	0	0
Empty	132	156	178	242	265	286

Table 6.5: Method 3b prediction structure breakdown showing singleton and abstain behaviour across α .

Method 3b achieves consistently better accuracy than Method 3a across all α values (69.64% vs 59.33% at $\alpha = 0.60$), while also maintaining substantially lower false alarm rates (28.45% vs 45.73%). The improvement is driven by a more nuanced decision mechanism: instead of conservatively flagging uncertain cases as misbehaviour, the method abstains on genuinely ambiguous inputs. This reduces over-penalisation of correct model outputs while still preserving meaningful failure detection. At $\alpha = 0.60$, coverage is 56%, meaning the method still makes predictions on a majority of inputs rather than collapsing into excessive abstention.

A notable pattern emerges in the accuracy curve. Performance remains relatively stable between $\alpha = 0.60$ and $\alpha = 0.80$ (69–72%), but degrades at $\alpha = 0.91$ and $\alpha = 0.95$ before partially recovering at $\alpha = 0.99$. This non-monotonic behaviour is consistent with calibration instability arising from the small calibration set. Since class-conditional thresholds are estimated using only approximately 13–14 examples per class (given $n = 90$), the resulting quantiles are highly sensitive to individual calibration points. Small changes in these few samples can shift thresholds disproportionately, producing unstable downstream selection behaviour.

An additional structural observation is that $t(1)$ is consistently higher than $t(0)$ across all α values. This indicates that the model requires stronger confidence to classify an example as correctly handled (label 1) than as misbehaviour (label 0). This asymmetry is likely driven by class imbalance in the data (199 label-1 vs 101 label-0 in the test set), which biases the calibration procedure toward stricter acceptance of the majority class. As a result, the decision boundary for label 1 becomes more conservative, effectively tightening the conditions under which the model is allowed to assert correctness.

6.6 Method 4 : Conformal Risk Control

Method 4 directly targets the metric that is most critical in safety-sensitive deployment: the false negative rate (FNR). Instead of optimising for overall accuracy, it performs a threshold sweep over the calibration set and selects the threshold that best matches a desired target FNR. This reframes the problem from “how accurate is the model overall?” to “how reliably can we avoid missing failures?”, which is a more appropriate objective in risk-sensitive settings.

Target FNR	Threshold t	Actual FNR	Accuracy	False Alarm Rate	FN	FP
0.05	0.0100	20.79%	57.00%	54.27%	21	108
0.10	0.0100	20.79%	57.00%	54.27%	21	108
0.15	0.0100	20.79%	57.00%	54.27%	21	108
0.20	0.0297	30.69%	59.33%	45.73%	31	91
0.30	0.0839	43.56%	59.33%	39.20%	44	78
0.40	0.4237	55.45%	64.00%	26.13%	56	52

Table 6.6: Threshold tuning results for target failure miss rate (FNR) settings.

A striking outcome is that the targets 0.05, 0.10, and 0.15 all produce identical results. This is not a computational error but a direct consequence of calibration set limitations. With only approximately 15 true misbehaviour examples in the calibration split (15% of the 101 training negatives), the empirical FNR can only be estimated in coarse increments of roughly $1/15 \approx 6.7\%$. As a result, the threshold sweep cannot achieve FNR levels as low as 5%, 10%, or 15% on the calibration data. The minimum achievable empirical FNR is already above these targets, so all three settings collapse to the same optimal threshold ($t = 0.01$), which effectively corresponds to a near-trivial classifier that flags almost all inputs.

The actual test-time FNR (20.79%) is substantially higher than even the most lenient target of 15%. This discrepancy highlights a clear calibration generalisation gap: the small calibration set does not provide a stable enough estimate of the threshold required to control FNR on unseen data. In other words, the calibration procedure is internally consistent but externally unreliable due to limited sample size and distribution shift between calibration and test sets.

Despite this limitation, Method 4 remains valuable. Even though the formal conformal-style guarantees are not fully realised at this scale, the method still produces a meaningful trade-off curve between false negatives and false positives. For example, at the lowest target setting, it achieves a 20.79% FNR with a 54.27% false alarm rate, which improves failure detection

compared to several baseline methods at comparable operating points. The key insight is that the underlying mechanism is valid; the failure lies not in the approach itself but in insufficient calibration data to support reliable statistical guarantees.

6.7 Method 5 : Misbehaviour Direction + CP Rule

Method 5 is the most geometrically motivated approach. Instead of training a logistic regression classifier, it constructs a single direction vector by taking the difference between the mean hidden state of misbehaving examples and the mean hidden state of correct examples. This defines a “misbehaviour direction” in the 3072-dimensional hidden state space, representing the primary axis along which representations shift from correct behaviour toward misbehaviour.

The magnitude of this separation (the L2 norm of the difference-in-means vector before normalisation) is 13.83, indicating a substantial distance between the two class centroids in hidden-state space. This suggests that the representation space is meaningfully structured with respect to the task. The training distribution (visualised in Figure 6.2) shows misbehaviour scores centred around +9.11 and safe scores centred around -4.72, with noticeable but incomplete separation. The test distribution preserves this structure but reveals an important limitation: a non-trivial portion of misbehaviour examples still receive negative scores and fall into the “safe” region. This overlap directly contributes to elevated false alarm rates under stricter thresholds.

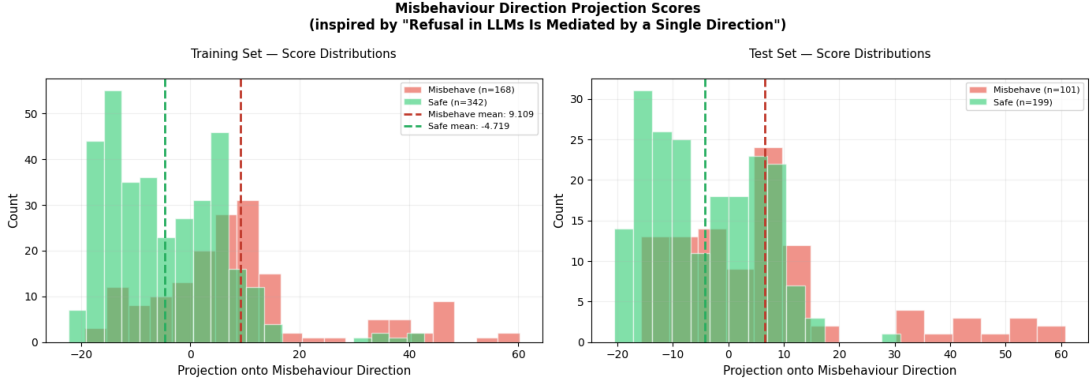


Figure 6.2: Misbehaviour Direction Projection Scores with a Separation of 13.8285

Alpha	0.60	0.70	0.80	0.91	0.95	0.99
Accuracy	58.67%	53.67%	48.33%	43.00%	36.33%	35.00%
Miss Rate	31.68%	12.87%	8.91%	1.98%	0.00%	0.00%
False Alarm Rate	46.23%	63.32%	73.37%	84.92%	95.98%	97.99%
FN	32	13	9	2	0	0
FP	92	126	146	169	191	195
Ambiguous	102/300	156/300	209/300	252/300	278/300	285/300

Table 6.7: Method 4 performance metrics across confidence levels α . Increasing α aggressively reduces failure miss rate at the cost of rapidly increasing false alarms and ambiguity.

Prediction Type	0.60	0.70	0.80	0.91	0.95	0.99
$q(\text{safe})$	-8.2677	-8.5533	-11.5888	-17.5261	-31.2014	-32.9007
$q(\text{misbehave})$	-2.9778	-10.1316	-12.5873	-15.6659	-18.4427	-19.6481
Empty	102	156	209	252	278	285
Singleton	198	144	91	48	22	15
Both	0	0	0	0	0	0

Table 6.8: Method 4 prediction structure breakdown showing quantile thresholds and how decisions collapse into empty (abstain) vs singleton predictions as α increases.

Method 5 achieves the strongest failure detection performance across all methods. At $\alpha = 0.91$, only 2 failures are missed (1.98% FNR), and at $\alpha = 0.95$, the failure miss rate drops to 0.00%, meaning every misbehaviour in the test set is successfully detected. This is a unique result among all evaluated methods, including logistic regression baselines, and suggests that the difference-in-means direction captures a highly stable geometric feature of misbehaviour in hidden-state space. At $\alpha = 0.60$, the miss rate is 31.68%, which, while still non-trivial, already improves significantly over the raw probe baseline (57.43%) and is comparable to Method 3a at the same operating point.

However, this zero-miss regime comes at a severe cost. At $\alpha = 0.95$, the false alarm rate rises to 95.98%, meaning almost every safe example is incorrectly flagged. As shown in Table 6.8, increasing α rapidly collapses the set of singleton predictions, while the ambiguous and empty regions dominate the output space. By $\alpha = 0.95$, 278 out of 300 examples are classified as either ambiguous or empty and are therefore conservatively flagged as misbehaviour. In effect, the decision rule becomes dominated by abstention-based pessimism, prioritising failure avoidance at the expense of usability.

The high false alarm rate relative to logistic regression methods arises from both geometric and

structural factors. Geometrically, the method enforces a single linear decision boundary orthogonal to the misbehaviour direction, which is more constrained than logistic regression, where the decision surface can rotate to better fit class boundaries. Structurally, the conservative rule amplifies ambiguity: any non-singleton prediction is treated as misbehaviour. Since Method 5 produces a larger proportion of ambiguous outputs due to independently calibrated class thresholds, it systematically triggers more conservative flags than Method 3a. This interaction between geometric constraint and conservative aggregation explains the extreme trade-off observed in the results.

6.8 Cross-Method Comparison at Alpha = 0.91

Figure 6.3 highlights a clear separation between three behavioural regimes at $\alpha = 0.91$: full-coverage predictors (M1, M3a, M4, M5), abstention-heavy conformal methods (M2, M3b), and their implications for operational reliability.

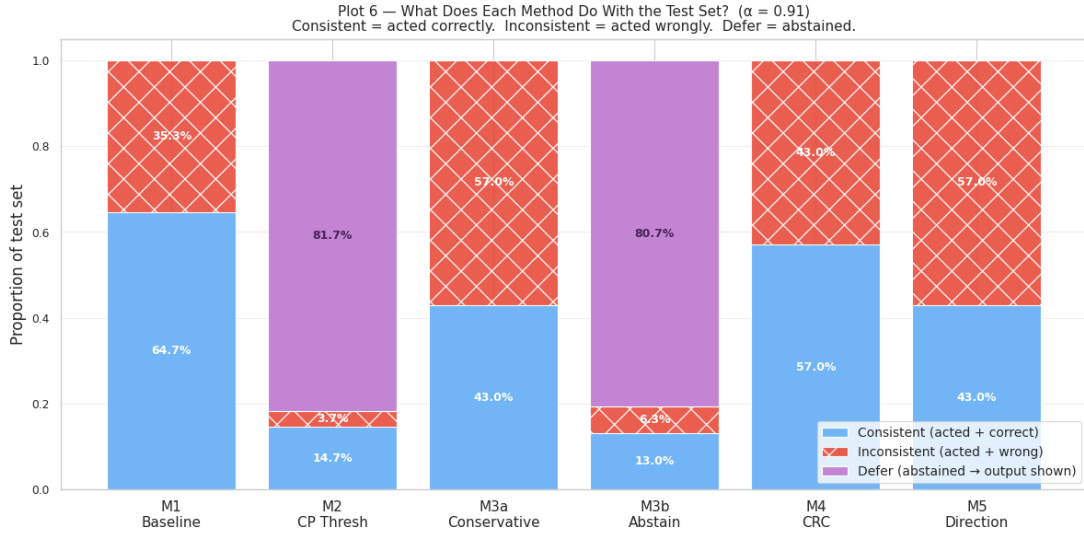


Figure 6.3: Decomposition of test set outcomes at alpha=0.91.

The most immediate observation is the near-identical behaviour of Methods 2 and 3b in this regime. Both methods abstain on roughly 80–82% of the test set (81.7% vs 80.7%), and among the small subset of retained predictions, they exhibit similar decomposition into correct and incorrect outcomes (M2: 14.7% consistent, 3.7% inconsistent; M3b: 13.0% consistent, 6.3% inconsistent). This similarity suggests that, at this operating point, both methods are effectively governed by the same underlying thresholding geometry, where only a narrow band of high-confidence singleton predictions survives calibration. In practice, this results in extremely

limited coverage and high abstention dominance, making both methods functionally similar despite their different formulations.

Method	Test Accuracy	Abstention Rate	Failure Miss	False Alarm
Baseline (M1)	64.67%	0.00%	57.43%	24.12%
CP Threshold (M2)	80.00%	81.67%	43.75%	10.26%
Conservative (M3a)	43.00%	0.00%	6.93%	82.41%
Class-Conditional (M3b)	67.24%	80.67%	23.81%	37.84%
CRC (M4)	57.00%	0.00%	20.79%	54.27%
Direction CP (M5)	43.00%	0.00%	1.98%	84.92%

Table 6.9: Comparison of all methods at $\alpha = 0.91$.

In contrast, Methods 1, 3a, 4, and 5 operate in a full-coverage regime, where every example is assigned a prediction. Within this group, performance diverges substantially depending on how uncertainty is handled. The raw baseline (M1) achieves the highest apparent consistency at 64.7%, but this comes with a high failure miss rate and no mechanism for uncertainty control. M3a and M5 exhibit identical behaviour in aggregate consistency (43.0% correct, 57.0% incorrect), reflecting the fact that both methods aggressively prioritise failure detection, but at the cost of substantially over-flagging safe behaviour.

Overall, the results at $\alpha = 0.91$ suggest a clear trade-off structure. Abstention-based conformal methods (M2, M3b) achieve higher selectivity but at the cost of minimal operational coverage, while full-coverage methods trade precision for continuous decision-making. The core tradeoff visible across all methods is that reducing miss rate almost always demands accepting more false alarms or sacrificing coverage, no single method dominates on all three axes simultaneously.

6.9 Visualizations

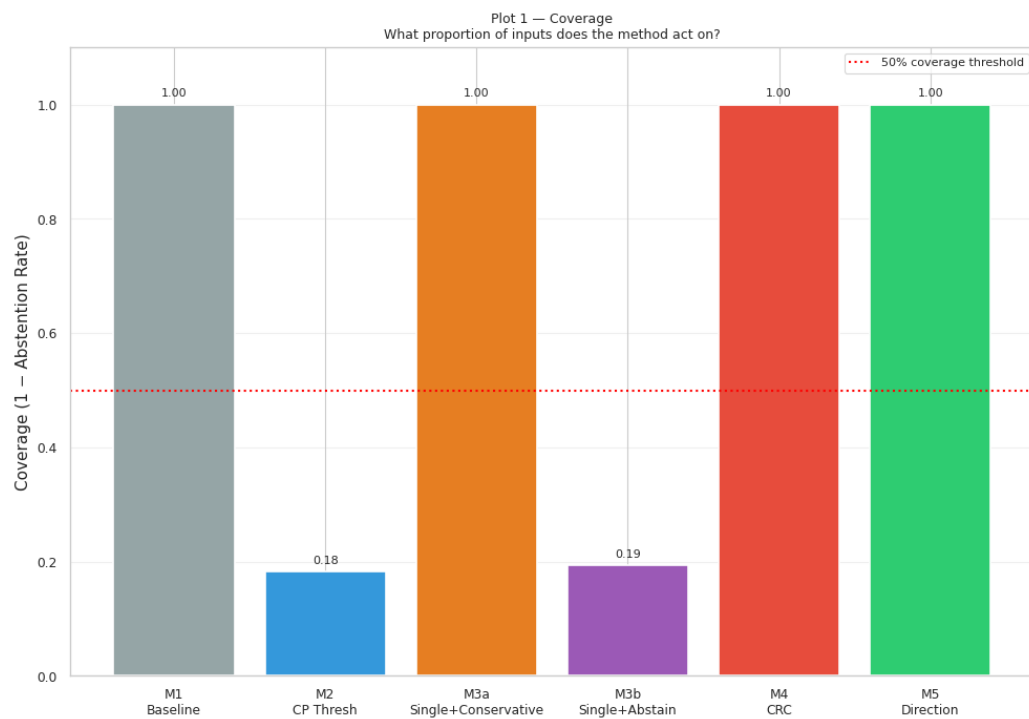


Figure 6.4: Coverage

Proportion of test inputs each method acts on ($1 - \text{abstention rate}$) at $\alpha = 0.91$

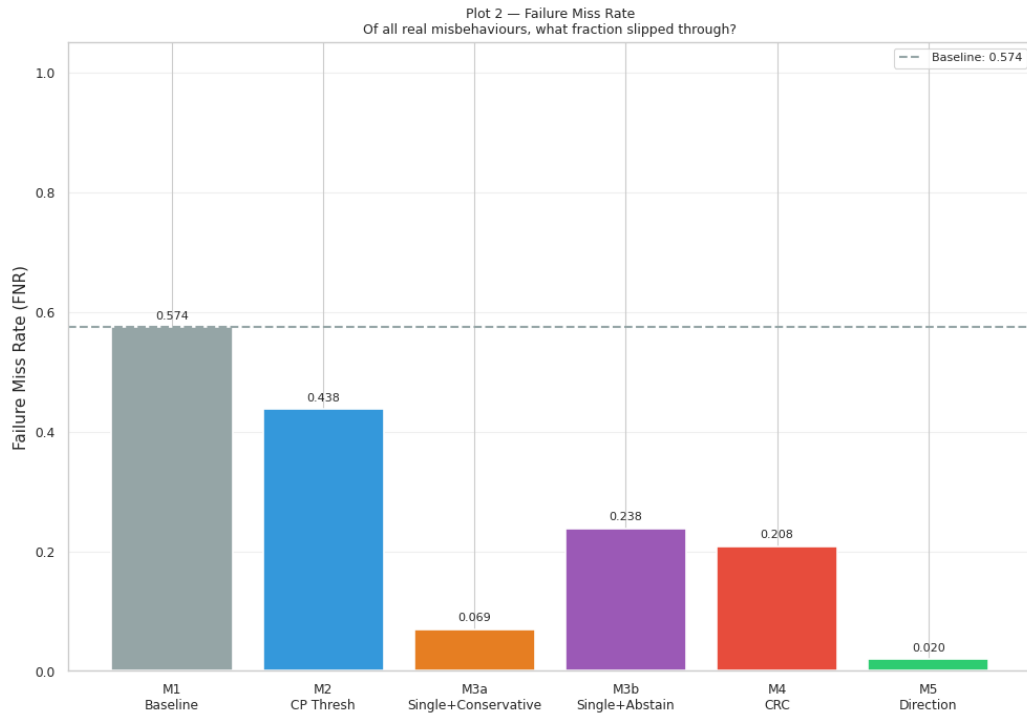


Figure 6.5: Failure Miss Rate
Of all genuine misbehaviours in the test set, the fraction each method failed to detect

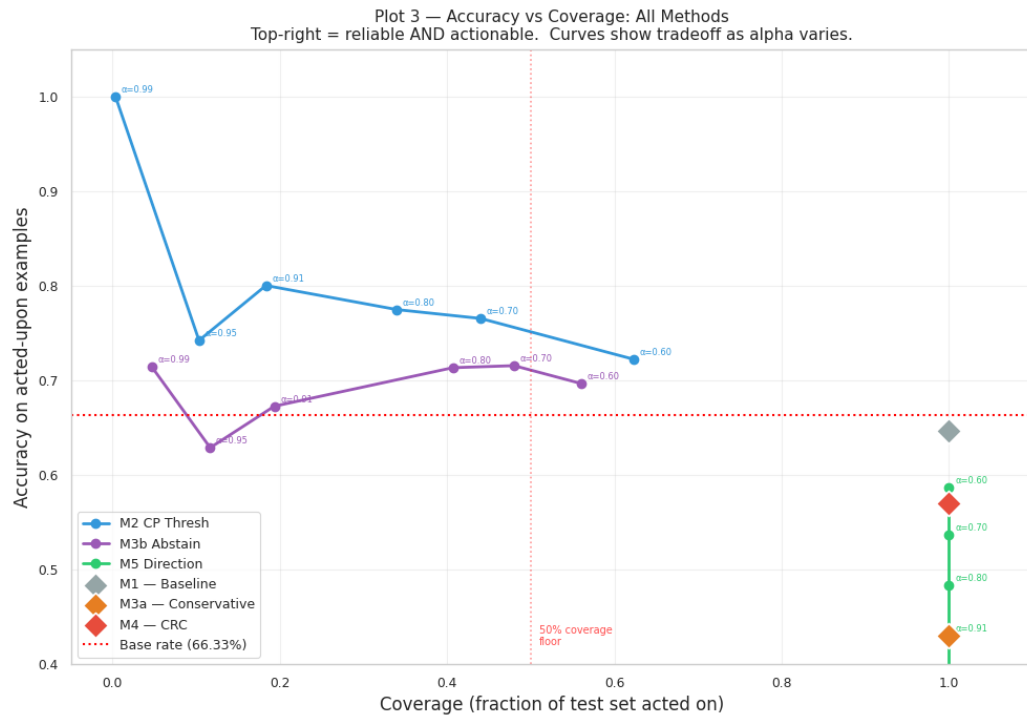


Figure 6.6: Accuracy vs Coverage
Each method's accuracy on the examples it acts on, plotted against the fraction of inputs it covers. The top-right corner is ideal.

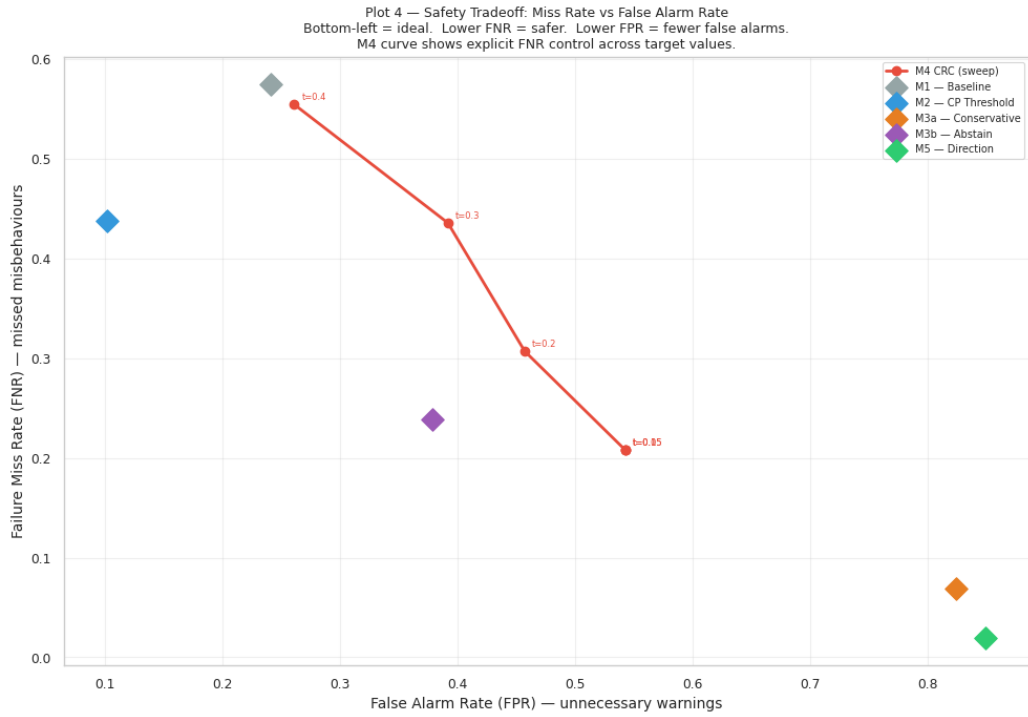


Figure 6.7: Safety Tradeoff (FNR vs FPR)
 Miss rate against false alarm rate for each method. The bottom-left is ideal.

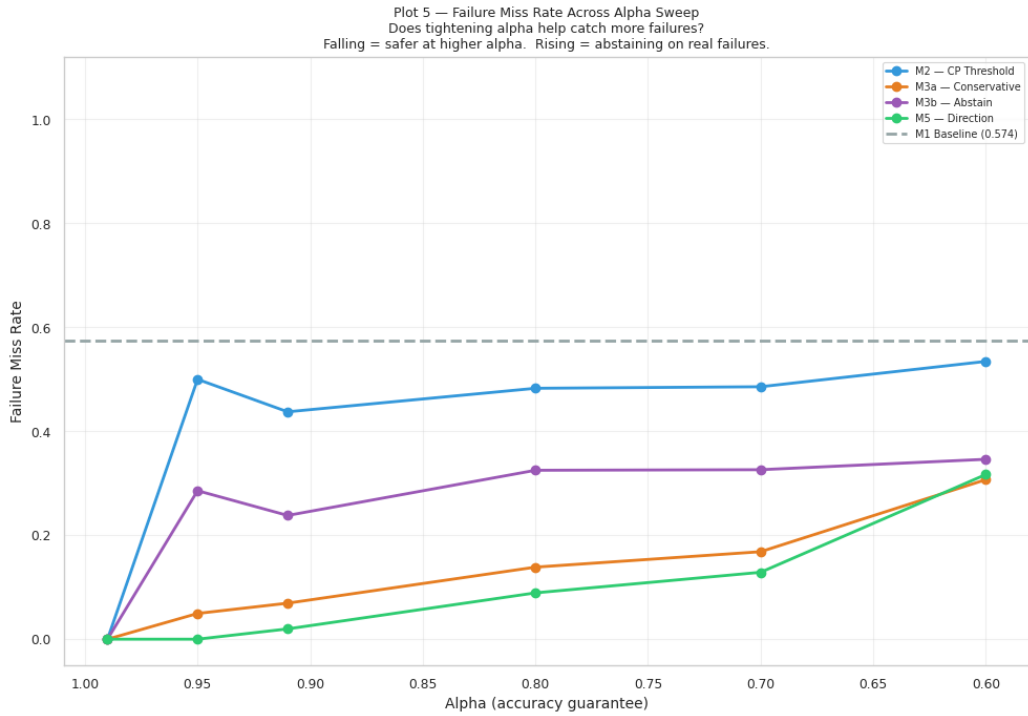


Figure 6.8: Failure Miss Rate Across Alpha Sweep
 How each method's miss rate changes as α tightens from 0.60 to 0.99.

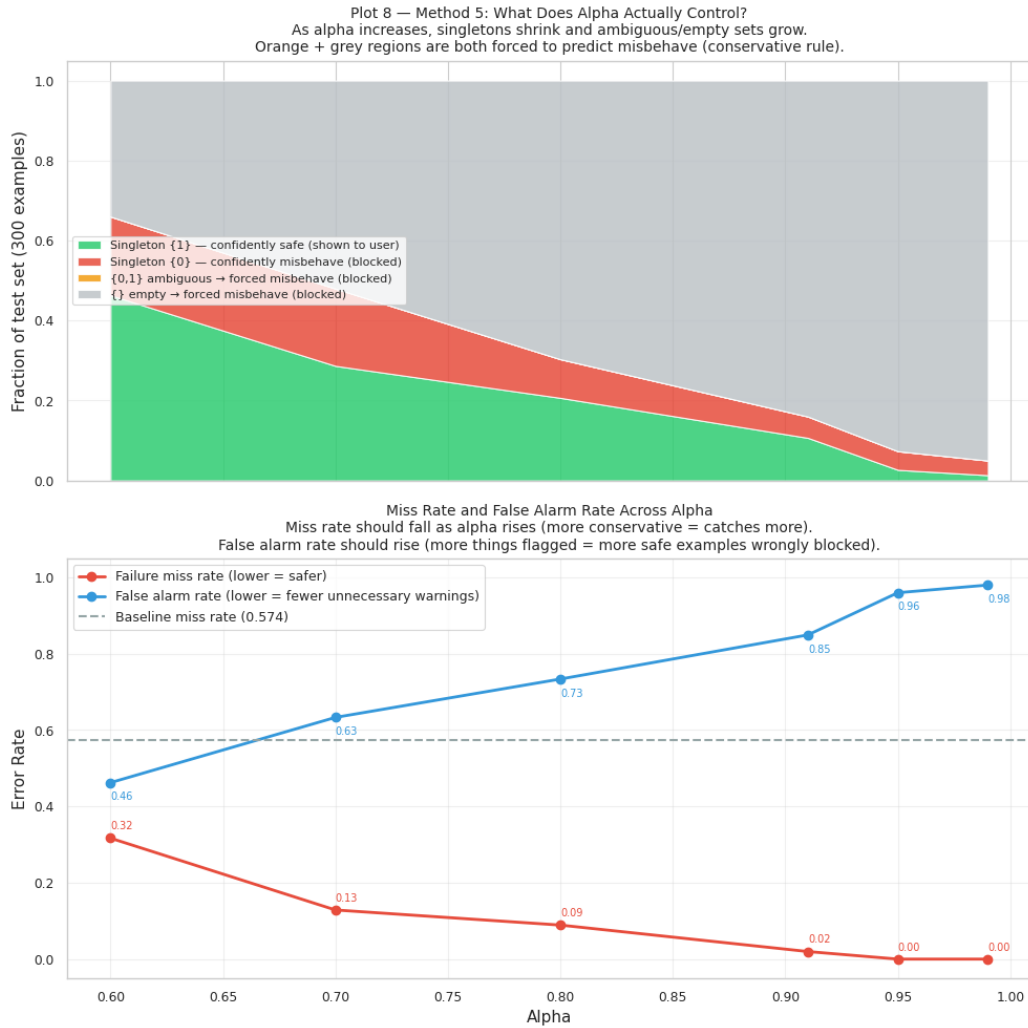


Figure 6.9: Method 5: What Alpha Actually Controls

Top panel shows how the composition of prediction sets changes with α , while the bottom panel tracks the resulting miss rate and false alarm rate.

Chapter 7

Legal, Social, Ethical and Professional Issues

This chapter evaluates the legal, social, ethical, and professional issues arising from this research, and demonstrates adherence to the British Computing Society (BCS) Code of Conduct and Code of Good Practice.

7.1 Legal Issues

This research uses the SelfAware dataset, a publicly available dataset, meaning no personal data is collected, stored, or exposed. As no personally identifiable information is processed, this research is compliant with the General Data Protection Regulation (GDPR). All models and probing classifiers were developed in a controlled research environment and are not deployed in any live system.

7.2 Ethical and Social Issues

The goal of this research is to better understand the internal representations of language models in order to improve their safety and reliability. While probing techniques could in principle be misused, the intent here is constructive: to surface potential failure modes before deployment. All experiments were conducted objectively, with care taken to avoid misrepresentation of results. Dataset selection was performed with attention to fairness and impartiality.

7.3 Professional Issues

All third-party resources, datasets, and libraries used in this project have been appropriately cited and attributed. The research was conducted within the bounds of professional competence, and findings are reported as observed without manipulation or embellishment.

7.4 BCS Code of Conduct

This project was conducted in accordance with the BCS Code of Conduct. In particular:

- **Public interest (1.a):** The research aims to improve the safety of language model systems, with due regard for public wellbeing and security.
- **Honesty and integrity (3.e):** Results are presented as found and have not been misrepresented.
- **Professional competence:** All work falls within the researcher’s area of competence, and limitations are acknowledged where appropriate.

Chapter 8

Conclusion and Future Work

This project investigated whether the internal activations of a large language model encode awareness of the limits of its own knowledge, using Llama-3.2-3B as the experimental subject and conformal prediction as the statistical framework for wrapping uncertainty guarantees around a linear probe trained on hidden states. The answer to the research question is a qualified yes: hidden states at layer 14 do geometrically separate correct from misbehaving responses, with a direction separation of 13.83, and Method 5 demonstrates that this separation is exploitable without any weight optimisation, achieving zero miss rate at $\alpha = 0.95$ using only the vector between class means. This is meaningful evidence that epistemic self-awareness is, to some degree, linearly encoded in Llama-3.2-3B’s representation space.

The key technical findings can be stated concisely. First, conformal prediction provides a principled and deployable framework for uncertainty-aware misbehaviour detection, but reliable coverage guarantees require calibration sets of at least 200+ examples, the $n = 90$ calibration split used here is insufficient, causing Method 4 to miss its FNR targets and Method 2 to show non-monotone accuracy-coverage curves. Second, the choice of deployment strategy matters more than the choice of conformal method: at $\alpha = 0.91$, the difference in false alarm rate between the best and worst full-coverage methods spans from 7% (M3a) to 85% (M5), a gap that dwarfs any accuracy difference. Third, no single method dominates on coverage, miss rate, and false alarm rate simultaneously, the optimal method depends entirely on whether the application treats missed misbehaviours or unnecessary interventions as the costlier error. Fourth, a correctness-based label design derived from ground-truth epistemic accuracy is a substantially more honest evaluation target than format-based proxies, though it introduces label noise that weakens the probe signal.

The comparison with the paper’s benchmarks reveals a consistent scale gap: the paper reports 91%+ conformal accuracy at 40–97% coverage using 8B–70B models on 5000+ training examples, whereas this experiment achieves at best 80% accuracy at 18.3% coverage with a 3B model and 600 training examples. This gap is not a failure of the approach, it validates the paper’s central prediction that probing effectiveness scales with model size but it does confirm that 3B parameters is below the threshold at which the signal becomes practically useful without substantially more data.

Several directions for future work follow directly from these findings. Scaling to Llama-3.1-8B is the most important single experiment: if direction separation and conformal accuracy improve markedly at 8B, it would confirm that the weak signal observed here is a scale artefact rather than a fundamental limitation of the probing approach. Expanding the training dataset to 2000+ examples would simultaneously improve probe generalisation and push calibration set size above the 200-example threshold required for reliable conformal guarantees. Inspecting the highest-magnitude dimensions of the Method 5 direction vector would identify which components of the network’s residual stream carry knowledge-limitation signals, potentially enabling targeted interpretability work. Finally, testing across multiple layers rather than fixing layer 14 would determine whether epistemic self-awareness is localised or distributed across the network depth, a question with direct implications for mechanistic interpretability research.

The broader implication of this work is that LLM misbehaviour detection does not require external ground-truth labels at inference time. A probe trained on hidden states, wrapped in a conformal prediction set, can provide statistically grounded uncertainty signals using only the model’s own internal representations. At sufficient scale and with adequate calibration data, this framework offers a viable path toward deployable, interpretable, and theoretically sound oversight of LLM behaviour - a goal that remains as important as it is technically difficult.

Chapter 9

References

[1] Ashok, D., & May, J. (2025) "Language Models Can Predict Their Own Behavior" Presented at the Thirty-Ninth Annual Conference on Neural Information Processing Systems (NeurIPS 2025). arXiv:2502.13329 [cs.CL].

URL : <https://doi.org/10.48550/arXiv.2502.13329>. (Accessed: 23 April 2026)

[2] Arditi, A., Obeso, O., Syed, A., Paleka, D., Panickssery, N., Gurnee, W., & Nanda, N. (2024) "Refusal in Language Models Is Mediated by a Single Direction" arXiv:2406.11717 [cs.LG].

URL : <https://doi.org/10.48550/arXiv.2406.11717>. (Accessed: 23 April 2026)

[3] Marks, S., & Tegmark, M. (2024) "The Geometry of Truth: Emergent Linear Structure in Large Language Model Representations of True/False Datasets" In Conference on Language Modeling (COLM 2024). arXiv:2310.06824 [cs.AI].

URL : <https://doi.org/10.48550/arXiv.2310.06824>. (Accessed: 23 April 2026)